



Nombre: Christopher Abraham Chairez Cruz Pablo Ángel Garza Ramírez Rubén Miguel Manzanares Tovar Mario Ángel García Canizales Luis Carlos Soto Carrillo	Matrícula: 3038149 7094461 2886925 3065072 7039730
Nombre del curso: Ingeniería en Desarrollo de Software	Nombre del profesor: Carlos Abraham Carballo Monsivais
Módulo: 1	Actividad: Proyecto Final
Fecha: 27/09/26	
Bibliografía: Repositorio del proyecto: https://github.com/mariocanizales777-lab/GestonHospitalaria <i>Referencias completas al final del documento.</i>	

FarmaCitas Fantásticas

Sistema de Gestión de Citas para Consultorios en Farmacias

Informe final del proyecto integrador

Índice

1. Resumen ejecutivo

Propósito del sistema

FarmaCitas Fantásticas es una plataforma web para gestionar las citas médicas de los consultorios de farmacia. Sustituye el registro manual en papel por un sistema digital en el que los pacientes consultan la disponibilidad, reservan, consultan y cancelan sus citas, revisan el catálogo de medicamentos y sus recetas. Por su parte, los doctores gestionan su agenda y emiten recetas, y el administrador da de alta a los doctores, publica horarios, controla el inventario y consulta reportes. El objetivo es reducir los tiempos de espera y las inasistencias, ordenar la carga de trabajo del personal y dar transparencia sobre la disponibilidad de medicamentos.

Alcance de esta entrega

Esta entrega integra las actividades previas del proyecto (Carta del Proyecto, Planeación y Arquitectura 4+1) con la implementación real del sistema. Cubre los siguientes componentes:

- **Módulo básico implementado:** autenticación y gestión de usuarios (registro, inicio de sesión y administración de doctores por rol), junto con el flujo principal de agendamiento de citas.
- **Seguridad:** autenticación con JWT (vigencia de 8 horas), contraseñas cifradas con bcrypt, autorización por roles (admin, doctor, paciente) y secretos en variables de entorno.
- **Pruebas:** pruebas unitarias con Jest, con 94.15% de cobertura de líneas, y 15 pruebas de API en Postman, incluyendo casos negativos 401 y 403.
- **CI/CD:** pipeline de GitHub Actions que prueba, compila y despliega automáticamente el backend en Render y el frontend en Vercel.
- **Escanero:** análisis de seguridad con OWASP ZAP dentro del pipeline y análisis de calidad de código con SonarQube Cloud.
- **Cierre:** comparación entre lo planificado y lo ejecutado, lecciones aprendidas y plan de mejora continua.

Resultados principales

Indicador	Resultado
Historias de usuario completadas	7 de 7 (HU1 a HU7), con 100% de sus criterios de aceptación
Cobertura de pruebas unitarias	94.15% de líneas y 100% de funciones (meta ≥ 80%)
Pipeline CI/CD	4 jobs: pruebas del backend, build del frontend, despliegue y escaneo OWASP ZAP
Frontend en producción	https://geston-hospitalaria.vercel.app
Backend en producción	https://farmacitas-backend.onrender.com
OWASP ZAP	Sin alertas de riesgo alto; 3 hallazgos de riesgo medio atendidos
SonarQube	Quality Gate no aprobado; se documenta un plan de corrección (sección 4.3)
Repositorio	https://github.com/mariocanizales777-lab/GestonHospitalaria

2. Antecedentes y documentación base

Esta sección integra de forma explícita las actividades previas del proyecto: la Carta del Proyecto (Actividad 1), la Planeación con historias de usuario y Trello (Actividad 3) y la Arquitectura de Software con el modelo 4+1 (Actividad 4). Cada síntesis se actualizó con lo que realmente se construyó.

2.1 Carta del Proyecto (síntesis)

Nombre del proyecto	FarmaCitas Fantásticas
Producto	Sistema de Gestión de Citas para Consultorios en Farmacias
Fecha de elaboración	16 de agosto de 2026
Metodología	Scrum: desarrollo por sprints, con entregas graduales y adaptación a cambios
Aprobación	Afirmativa ficta: 3 días hábiles para emitir observaciones; sin comentarios, el documento se considera aprobado

Objetivo general

Desarrollar una plataforma para la gestión de citas en consultorios de farmacia que permita organizar de manera más eficiente el proceso de atención médica. El sistema facilita el registro, la administración y el seguimiento de las consultas, con el propósito de reducir los tiempos de espera, mejorar la organización del consultorio y brindar un servicio más ágil y accesible para pacientes y personal de la farmacia. En una primera etapa está pensado para farmacias de Nuevo León, con un diseño que permite expandirlo a otras regiones.

Alcance

Incluye	Excluye
Registro e inicio de sesión con roles (paciente, doctor, administrador)	Pagos en línea y facturación
Gestión de horarios y disponibilidad por sucursal (HU3)	Expediente clínico electrónico completo
Agendamiento de citas (HU1) y su consulta y cancelación (HU2)	Telemedicina o videoconsultas
Catálogo de medicamentos, recetas digitales y pedidos (HU5)	Venta y envío de medicamentos a domicilio
Interfaces por rol: paciente, doctor y administrador (HU6, HU7)	Integración con aseguradoras
Reportes para el administrador	Notificaciones por SMS o correo mediante proveedor externo (pospuesto)

Entregables

Tipo	Entregable
Módulos	API REST en Node.js + Express (módulos: auth, usuarios, horarios, citas, medicamentos, recetas, pedidos y reportes) y aplicación web en React + Vite con vistas por rol
Documentación	Carta del Proyecto, historias de usuario (INVEST y MoSCoW), tablero Kanban, arquitectura 4+1 con diagramas UML y este informe final
Despliegue	Frontend en Vercel, backend en Render, base de datos en MongoDB Atlas; Dockerfile y docker-compose para desarrollo local; pipeline CI/CD en GitHub Actions
Calidad y seguridad	Reporte de cobertura (Jest), colección de pruebas en Postman, reporte OWASP ZAP y métricas de SonarQube

Stakeholders

Stakeholder	Interés / rol
Pacientes	Agendar y administrar citas sin filas; consultar medicamentos y recetas
Doctores / personal de consultorio	Consultar su agenda, atender consultas y emitir recetas
Administrador del sistema	Gestionar doctores, horarios, inventario y reportes
Gerencia de la farmacia	Mejorar el servicio, reducir inasistencias y obtener ventaja competitiva
Equipo de desarrollo (5 integrantes)	Construir, probar, desplegar y documentar el sistema
Profesor (patrocinador académico)	Evaluar entregables y avance del proyecto
Proveedores de servicios en la nube	Vercel, Render, MongoDB Atlas y GitHub

Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Dos pacientes reservan el mismo horario al mismo tiempo	Alto	Validar en el backend que el horario siga disponible antes de registrar la cita y marcarlo como ocupado
Vulnerabilidades en datos de salud de los pacientes	Alto	JWT, contraseñas con bcrypt, control de acceso por rol, HTTPS y escaneos con OWASP ZAP y SonarQube
Resistencia al cambio del personal	Alto	Interfaz sencilla con pantallas por rol y capacitación previa al despliegue
Saturación en horarios de alta demanda	Alto	Servicios en la nube escalables y pruebas de carga antes del lanzamiento
Diferencia entre inventario del sistema y stock físico	Medio	Actualización de stock por el administrador y revisión física periódica
Alta tasa de inasistencia de pacientes	Medio	Recordatorios de citas en la interfaz (componente RecordatorioCitas)
Retrasos por carga académica del equipo	Alto	Sprints cortos, prioridad MoSCoW y seguimiento en Trello
Errores de captura de información	Bajo	Campos obligatorios y validaciones en modelos y formularios

Supuestos y restricciones

- Los consultorios cuentan con equipo de cómputo y conexión a internet durante su horario de operación.
- Los pacientes tienen acceso a un navegador web desde computadora o celular.
- El personal médico y administrativo estará disponible para capacitarse antes del despliegue formal.
- La normativa local sobre datos personales y de salud no cambiará de forma significativa durante el proyecto.
- La infraestructura en la nube estará disponible antes de las pruebas; se usan planes gratuitos de Vercel, Render y MongoDB Atlas.
- **Restricciones:** tolerancia de $\pm 10\%$ en tiempo y $\pm 5\%$ en costo; disponibilidad objetivo del 99% en horario de atención; respuesta menor a 2 segundos por transacción; cumplimiento obligatorio de la normativa de protección de datos.

2.2 Planeación del Proyecto (síntesis)

Historias de usuario

En la Actividad 3 se redactaron tres historias de usuario iniciales (HU1 a HU3), que después se refinaron con los criterios INVEST. Durante la ejecución surgieron cuatro historias más (HU4 a HU7) para cubrir la seguridad, las recetas y las interfaces por rol. La tabla resume las siete; la HU4 y la HU6 corresponden al módulo implementado que se documenta en la sección 3.

ID	Historia de usuario	Criterios de aceptación principales
HU1	Como paciente , quiero consultar los horarios disponibles y reservar una cita seleccionando sucursal, médico, fecha y hora, para asegurar mi atención y reducir el tiempo de espera.	Solo se muestran horarios disponibles; no se permite la doble reserva; la cita queda relacionada en la base de datos; se muestra una confirmación con los datos clave.
HU2	Como paciente , quiero consultar y administrar mis citas, para mantener mi agenda actualizada y cancelar o modificar una cita cuando sea necesario.	El paciente solo ve sus propias citas; cancelar libera el horario; los cambios se reflejan en la disponibilidad; los estados se muestran con etiquetas de color.
HU3	Como personal del consultorio , quiero administrar los horarios de atención de los médicos, para mantener actualizada la disponibilidad que ven los pacientes.	Solo personal autorizado registra horarios; cada horario tiene médico y sucursal; no se permiten duplicados ni horarios en el pasado.
HU4	Como usuario del sistema , quiero iniciar sesión con mis credenciales y tener accesos delimitados según mi rol, para resguardar la seguridad de la información.	Credenciales válidas generan un JWT; credenciales incorrectas devuelven 401; rutas no autorizadas devuelven 403; registro con usuario único y contraseña cifrada.
HU5	Como profesional de la salud o paciente , quiero consultar el catálogo de medicamentos y emitir o revisar recetas digitales, para dar seguimiento a los tratamientos.	El médico asocia medicamentos a una receta; el paciente solo ve sus recetas; el catálogo tiene datos iniciales; los endpoints de recetas tienen pruebas automatizadas.

ID	Historia de usuario	Criterios de aceptación principales
HU6	Como usuario (paciente, doctor o administrador) , quiero registrarme, iniciar sesión y tener una navegación adaptada a mi rol, para acceder de forma intuitiva a mis funciones.	Login con validación de campos; formulario de registro; barra de navegación dinámica por rol; pantallas de inicio diferenciadas.
HU7	Como paciente o doctor , quiero agendar, revisar y gestionar mis citas desde una interfaz interactiva, para llevar un control eficiente de mi agenda.	Formulario para agendar; vista de citas del paciente; agenda diaria del médico; visualización de bloques de horario disponibles.

Priorización MoSCoW

Se usó MoSCoW para clasificar las historias según su criticidad y garantizar un producto mínimo viable funcional desde el primer despliegue.

Prioridad	Historias	Justificación
Must have	HU4 – Autenticación y control de acceso HU6 – Interfaz de autenticación y navegación por rol HU3 – Gestión de horarios HU1 – Agendamiento de citas	Sin seguridad no se puede operar con datos de salud; sin horarios no hay nada que reservar; el agendamiento es la propuesta de valor principal.
Should have	HU7 – Interfaz de agendamiento y gestión HU2 – Consulta y administración de citas	Completan el ciclo de vida de la cita y dan la usabilidad necesaria para pacientes y doctores.
Could have	HU5 – Medicamentos y recetas	Aporta valor médico adicional, pero puede habilitarse después sin impedir el agendamiento.
Won't have (por ahora)	Pagos en línea, telemedicina, notificaciones por SMS	Fuera del alcance de las 6 semanas del proyecto.

Verificación INVEST de la historia del módulo implementado (HU4)

Criterio	Cumplimiento
Independiente	Se desarrolla sin depender de otras historias; las demás dependen de ella
Negociable	El mecanismo (JWT, duración del token, roles) se acordó con el equipo
Valiosa	Protege los datos de salud y habilita las funciones de cada rol
Estimable	Acotada a una semana (10 – 16 de septiembre)
Pequeña	Cabe en un solo sprint
Testeable	Criterios verificables con códigos HTTP (200, 401, 403) probados en Postman

Tablero Kanban (Trello)

El tablero tiene cuatro columnas: **Backlog** (historias identificadas), **Todo** (planeadas para el sprint), **En revisión** (terminadas y en revisión de QA y del equipo) y **Done** (cumplen todos sus criterios). Cada tarjeta incluye descripción, responsables, fechas y un checklist que funciona como criterios de aceptación. No se

configuró un límite WIP formal, pero se trabajó con pocas historias activas a la vez. Al cierre, las **13 tarjetas** (7 historias de usuario, 5 tareas de documentación DOC-01 a DOC-05 y la tarea técnica DEV-03 de integración continua) están en Done.

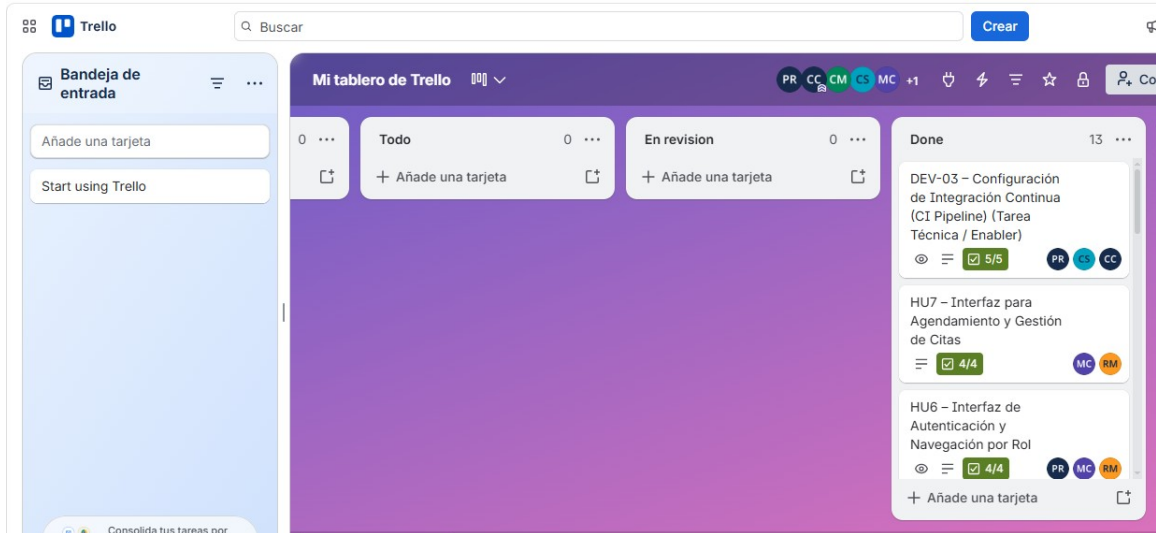


Figura 1. Tablero Kanban en Trello al cierre del proyecto (13 tarjetas en Done).



Figura 2. Tarjetas de historias de usuario y documentación con fechas, responsables y checklist completado.

Cronograma estimado vs. real

Actividad	Estimado (Act. 3)	Real (Trello)	Desviación
DOC-01 – Carta del proyecto y gestión inicial	—	16 – 17 ago	—
DOC-02 – Historias de usuario (INVEST)	—	23 ago	—
DOC-03 – Priorización y planificación	—	25 ago	—
DOC-04 – Arquitectura 4+1	—	26 – 28 ago	—
DOC-05 – Validación, pruebas y monitoreo	—	29 ago	—
HU3 – Gestión de horarios	24 – 29 ago	24 ago – 9 sept	+11 días
HU1 – Agendamiento de citas	7 – 18 sept	9 – 18 sept	Inicio +2 días; cierre en fecha
HU4 / HU6 – Autenticación (API e interfaz)	No planeada	10 – 16 sept	Alcance nuevo

Actividad	Estimado (Act. 3)	Real (Trello)	Desviación
HU5 – Medicamentos y recetas	No planeada	17 – 21 sept	Alcance nuevo
HU2 – Consulta y administración de citas	21 sept – 2 oct	21 – 27 sept	5 días antes
HU7 / Pedidos / Reportes – Interfaz	No planeada	22 – 25 sept	Alcance nuevo
DEV-03 – Integración continua (pipeline)	No planeada	Sept (tarea técnica)	Alcance nuevo

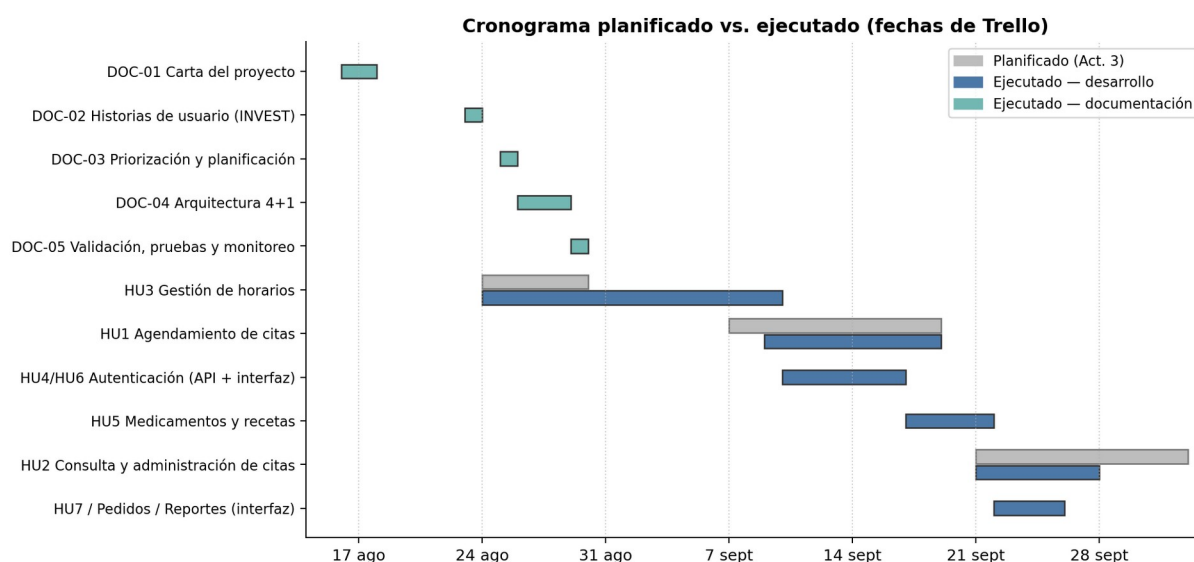


Figura 3. Cronograma planificado vs. ejecutado.

Historias implementadas y no implementadas

Las siete historias se completaron. Sin embargo, algunos elementos de los borradores de la Actividad 3 y del diseño de la Actividad 4 se pospusieron para una siguiente etapa:

Elemento	Estado	Comentario
HU1 a HU7	✓ Implementadas	Checklists de Trello al 100%
Bloqueo temporal de 10 minutos al reservar	Pospuesto	El horario pasa directamente de disponible a ocupado al confirmar
Recordatorio por SMS o correo 1 hora antes	Parcial	Recordatorios dentro de la interfaz; sin proveedor externo
Sugerencia automática de medicamentos genéricos	No implementado	Se priorizó el control de medicamentos controlados y las recetas
Reasignación automática a otra sucursal por ausencia del médico	No implementado	Propuesto para el plan de mejora
Tolerancia de 10 minutos y cancelación con 2 horas de anticipación	No implementado	Regla de negocio pendiente

2.3 Arquitectura de Software (síntesis 4+1 + UML)

Decisión arquitectónica implementada

En la Actividad 4 se propuso una arquitectura de microservicios como diseño objetivo. Para esta entrega, el equipo implementó un **monolito modular en capas**: una API REST en Node.js + Express conectada a MongoDB Atlas y una aplicación web en React + Vite. La decisión responde al tamaño del equipo (5 integrantes), a la duración de 6 semanas y a que el alcance actual no requiere escalar servicios por separado. Cada módulo del backend corresponde a un microservicio planeado, por lo que una migración futura implicaría mover módulos completos sin rediseñarlos.

Microservicio planeado (Act. 4)	Módulo implementado
svc-usuarios	auth + usuarios
svc-horarios	horarios
svc-citas	citas
svc-medicamentos	medicamentos + recetas + pedidos
svc-notificaciones	Recordatorios en la interfaz (RecordatorioCitas)
— (nuevo)	reportes (solo administrador)

1. Vista lógica (componentes / servicios)

El sistema se divide en tres bloques. El **frontend** agrupa las vistas por rol (paciente, doctor, administrador) y componentes comunes (Login, Registro, NavBarRol, Toast, ConfirmDialog); toda su comunicación pasa por `api.js`, que adjunta el token JWT. El **backend** recibe las peticiones en `app.js`, las filtra con el middleware de seguridad (`auth.js` y `requireRol.js`) y las envía al módulo correspondiente. Finalmente, los **modelos Mongoose** guardan la información en MongoDB.

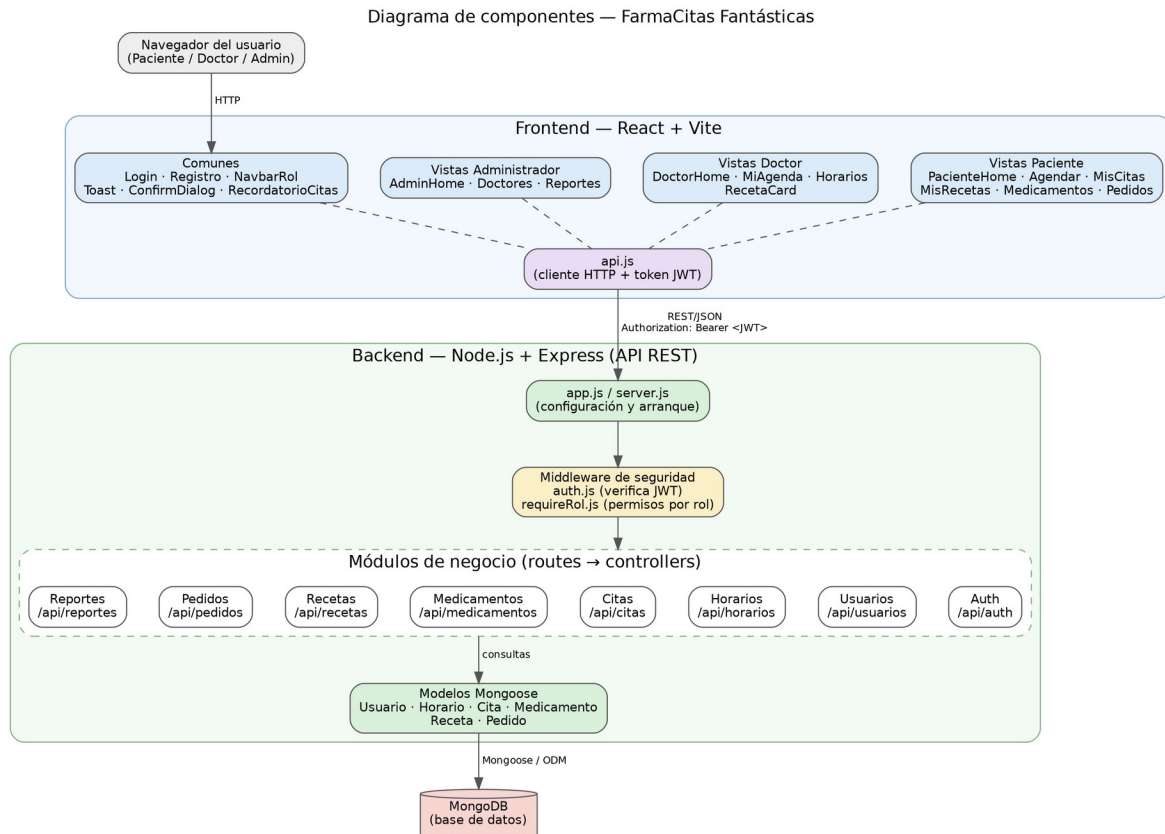


Figura 4. Diagrama de componentes (vista lógica).

2. Vista de desarrollo (módulos / paquetes)

El código vive en un solo repositorio (GestonHospitalaria) con las carpetas Backend y Frontend. El backend separa responsabilidades en routes (URLs), controllers (lógica de negocio), models (esquemas de MongoDB), middleware (seguridad), config (conexión y datos iniciales) y tests (Jest). El frontend organiza sus pantallas en components. En la raíz se encuentran docker-compose.yml y el pipeline .github/workflows/ci.yml. El trabajo se organizó en ramas de funcionalidad (por ejemplo, implementacionjest y pipeline-Frontend) que se integran a main mediante pull requests.

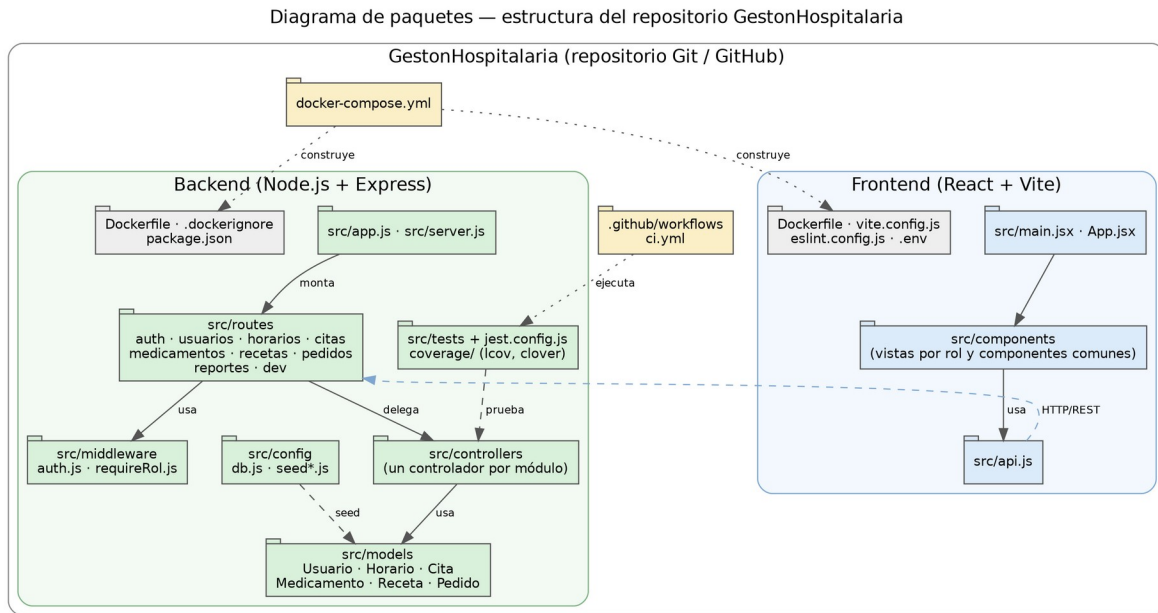


Figura 5. Diagrama de paquetes (vista de desarrollo).

3. Vista de procesos (interacciones y concurrencia)

La comunicación entre frontend y backend es **síncrona por REST/JSON sobre HTTPS**. Cada petición protegida pasa primero por la validación del token y después por la del rol. El punto crítico de concurrencia es la reserva: antes de registrar la cita, el backend verifica que el horario siga en estado disponible y lo cambia a ocupado, de modo que un segundo paciente ya no puede reservarlo. Las figuras 6 y 7 muestran los flujos de registro e inicio de sesión y de agendamiento.

Diagrama de secuencia – Registro de paciente e inicio de sesión

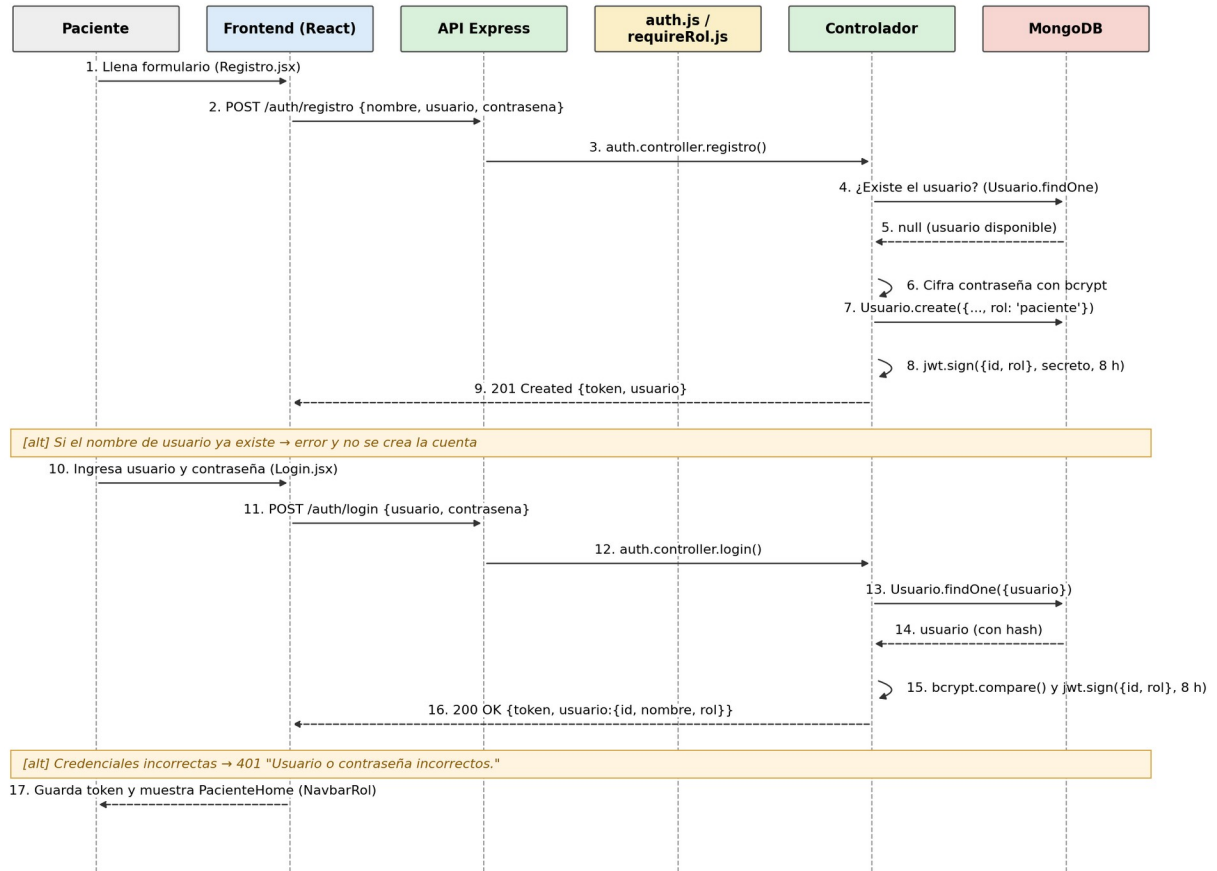


Figura 6. Diagrama de secuencia: registro de paciente e inicio de sesión.

Diagrama de secuencia — Agendar cita (módulo implementado, HU1)

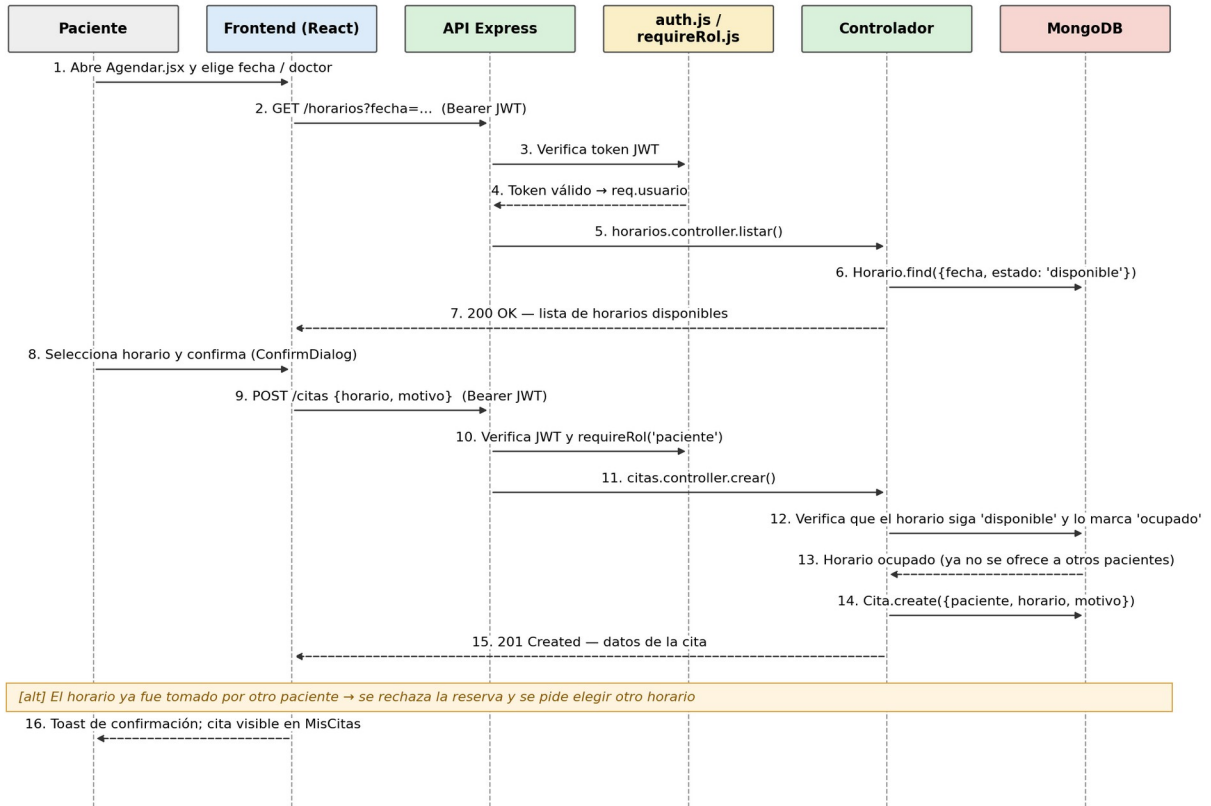


Figura 7. Diagrama de secuencia: agendar cita.

4. Vista física (despliegue)

En producción, el sistema usa tres servicios gestionados: el frontend se publica en **Vercel** como sitio estático, el backend corre en **Render** y los datos se guardan en **MongoDB Atlas**. La comunicación viaja por HTTPS y las peticiones protegidas llevan el token JWT. Los despliegues se activan desde GitHub Actions cuando un cambio llega a main. Para desarrollo local, el equipo usa **Docker Compose** con tres contenedores (frontend, backend y MongoDB con volumen persistente).

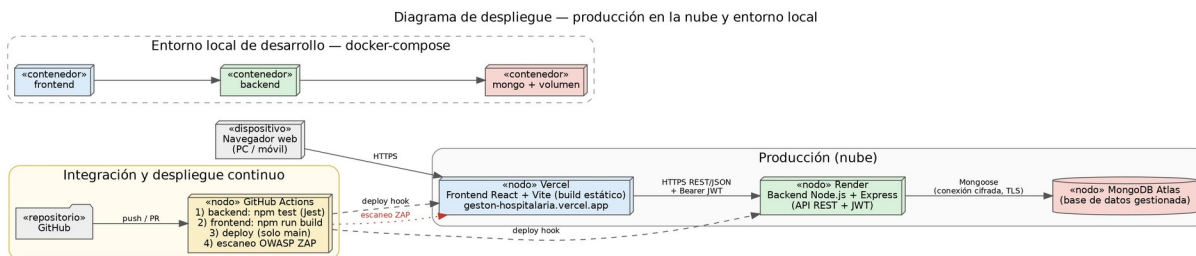


Figura 8. Diagrama de despliegue: producción en la nube y entorno local con contenedores.

Modelo de datos

MongoDB almacena seis colecciones relacionadas mediante referencias `ObjectId`. El rol del usuario es un campo del documento Usuario (paciente, doctor o admin).

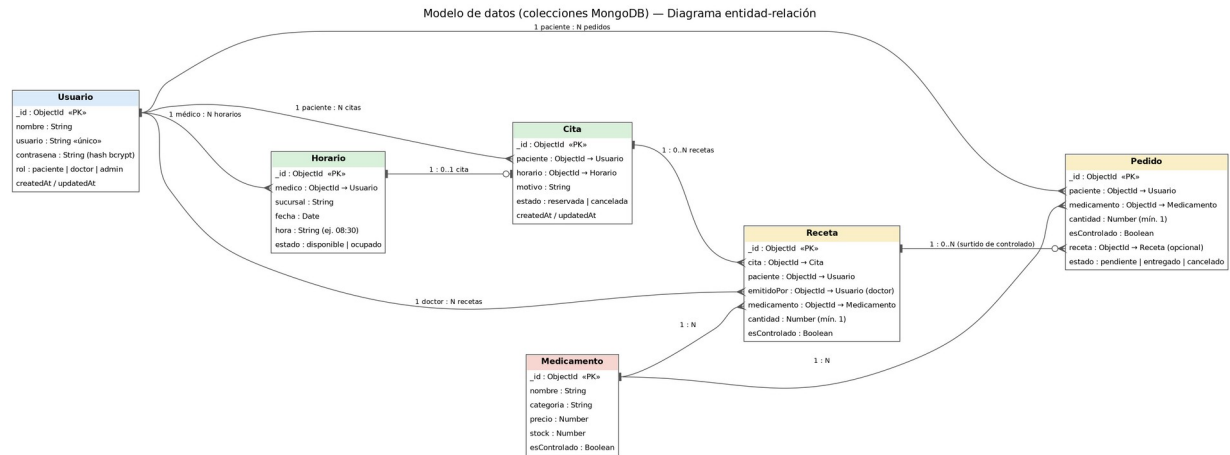


Figura 9. Diagrama entidad-relación de las colecciones de MongoDB.

+1. Escenarios (casos de uso clave)

Los tres actores del sistema son el paciente, el doctor y el administrador. Los escenarios clave son HU1 (agendar cita), HU2 (administrar citas) y HU3 (gestionar horarios), y todos requieren iniciar sesión (HU4).

Diagrama de casos de uso — FarmaCitas Fantásticas

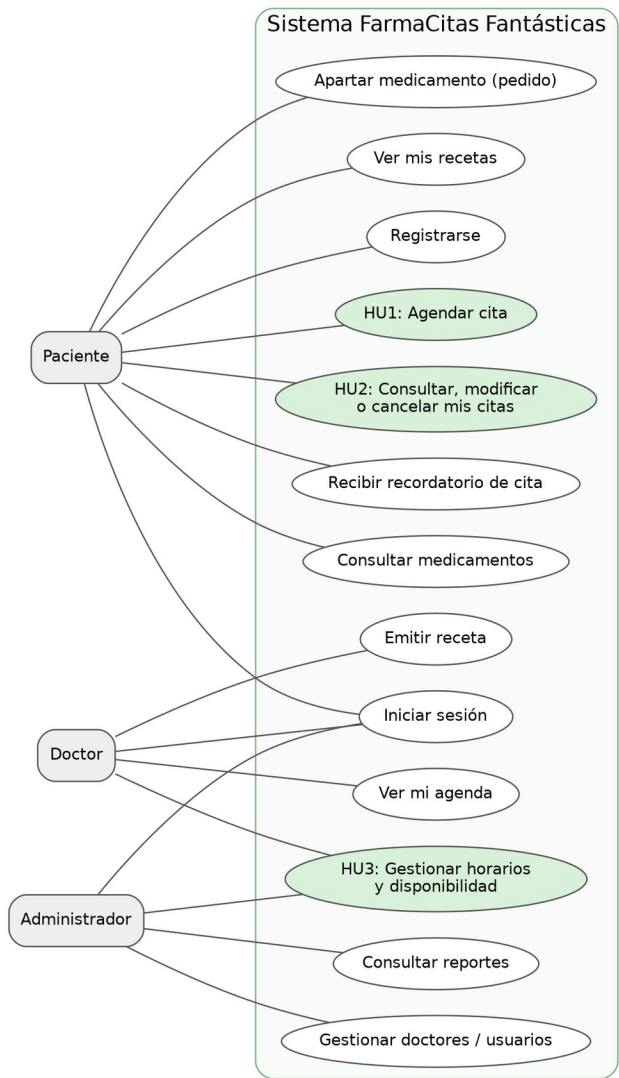


Figura 10. Diagrama de casos de uso.

Escenario narrativo – Agendar cita (HU1). Precondición: el paciente inició sesión. (1) El paciente abre la sección Agendar y elige fecha y doctor. (2) El sistema muestra solo los horarios en estado disponible. (3) El paciente selecciona un horario, escribe el motivo y confirma. (4) El backend valida el token y el rol, verifica que el horario siga disponible, lo marca como ocupado y registra la cita en estado reservada. (5) El sistema muestra la confirmación y la cita aparece en Mis Citas. Flujo alternativo: si otro paciente tomó el horario, la reserva se rechaza y se pide elegir otro.

Diagramas UML incluidos

Diagrama solicitado	Figura
Componentes o servicios	Figura 4 (componentes) y Figura 5 (paquetes)
Secuencia (login + registro)	Figura 6 (registro e inicio de sesión) y Figura 7 (agendar cita)

Diagrama solicitado	Figura
Despliegue (Docker / contenedores)	Figura 8
Entidad-relación	Figura 9
Casos de uso	Figura 10

Los diagramas se generaron con Graphviz y Matplotlib (Python) y se guardan en formato PNG en la carpeta /docs/arquitectura/ del repositorio.

3. Implementación y seguridad

3.1 Módulo implementado: autenticación y gestión de usuarios

Este módulo permite que los pacientes creen su cuenta e inicien sesión, y que el administrador dé de alta, consulte y elimine doctores. Es la base de todo el sistema, porque cada acción posterior (agendar, gestionar horarios, emitir recetas, actualizar stock) depende de saber quién es el usuario y qué rol tiene. Equivale al módulo de "registro de usuarios" que solicita la entrega.

Entidades principales

Entidad	Campos	Descripción
Usuario	_id, nombre, usuario (único), contraseña (hash bcrypt), rol, createdAt, updatedAt	Representa a cualquier persona que entra al sistema.
Rol	paciente · doctor · admin	Campo del Usuario (no es una colección aparte); define qué puede hacer cada quien y viaja dentro del token JWT.

Los pacientes se crean a sí mismos mediante el registro; los doctores los crea el administrador; y el administrador inicial se carga con el script `seedUsuarios.js`. El resto de las entidades del sistema (Horario, Cita, Medicamento, Receta y Pedido) se muestran en el diagrama entidad-relación de la sección 2.3.

Validaciones

Validación	Cómo se aplica
Campos obligatorios	Los esquemas Mongoose marcan como <code>required</code> los campos clave (por ejemplo, paciente, horario y motivo en Cita); el formulario de login valida los campos antes de enviarlos
Nombre de usuario único	No se permite registrar un usuario que ya existe
Valores permitidos	Estados con lista cerrada (enum): Cita reservada cancelada, Horario disponible ocupado, Pedido pendiente entregado cancelado
Rangos numéricos	cantidad en recetas y pedidos con valor mínimo de 1
Credenciales incorrectas	401 – "Usuario o contraseña incorrectos."
Petición sin token	401 – "Token requerido"
Token inválido o vencido	401 – "Token inválido o expirado"
Rol sin permiso	403 – "No tienes permiso para realizar esta acción"

Persistencia

Los datos se guardan en **MongoDB Atlas** (base de datos gestionada en la nube). El backend se conecta mediante **Mongoose** desde `config/db.js`, usando una cadena de conexión definida en variables de entorno. Los datos iniciales de usuarios y medicamentos se cargan con los scripts `seed.js`,

seedUsuarios.js y seedMedicamentos.js. En desarrollo local, MongoDB corre en un contenedor Docker con un volumen persistente.

Endpoints

Rutas reales relativas a la URL base del backend (<https://farmacitas-backend.onrender.com>):

Método	Ruta	Acceso	Qué hace	Respuesta probada
POST	/auth/registro	Público	Registra un paciente y devuelve su token	201 Created
POST	/auth/login	Público	Valida credenciales y emite el JWT	200 OK / 401
POST	/usuarios/doctores	Solo admin	Crea un doctor	201 / 401 sin token
GET	/usuarios?rol=doctor	Solo admin	Lista usuarios filtrados por rol	200 OK
DELETE	/usuarios/doctores/:id	Solo admin	Elimina un doctor	200 "Doctor eliminado"
POST	/horarios	Solo admin	Crea un horario	403 si no es admin
PUT	/horarios/:id	Solo admin	Edita un horario	200 OK
POST	/medicamentos	Solo admin	Registra un medicamento	201 Created
PUT	/medicamentos/:id	Solo admin	Actualiza el stock	200 OK
GET	/medicamentos	Autenticado	Consulta el catálogo	200 OK
GET	/citas/mi-agenda	Solo doctor	Consulta su agenda	200 OK

3.2 Autenticación JWT y roles

Flujo de autenticación

1. **Login:** el usuario envía usuario y contraseña a POST /auth/login.
2. **Verificación:** el backend busca al usuario en MongoDB y compara la contraseña con el hash guardado usando bcrypt.
3. **Emisión del token:** si las credenciales son correctas, genera un JWT firmado con el id y el rol del usuario, con vigencia de 8 horas, y lo devuelve junto con {id, nombre, rol}.
4. **Uso del token:** el frontend lo guarda y api.js lo envía en cada petición con el encabezado Authorization: Bearer <token>.
5. **Validación:** en cada ruta protegida, verificarToken (middleware auth.js) comprueba la firma y la vigencia del token; después, requireRol revisa que el rol tenga permiso para esa acción.

Autorización por roles

Rol	Acciones permitidas
Admin	Crear, listar y eliminar doctores; crear y editar horarios; registrar medicamentos y actualizar stock; consultar reportes
Doctor	Consultar su agenda (/citas/mi-agenda), revisar sus horarios y emitir recetas

Rol	Acciones permitidas
Paciente	Registrarse, iniciar sesión, agendar y cancelar sus citas, consultar medicamentos, hacer pedidos y ver únicamente sus recetas

Medidas mínimas de seguridad

Medida	Implementación
Hash de contraseñas	bcrypt; la contraseña nunca se guarda en texto plano ni se devuelve en las respuestas de la API
Expiración del token	8 horas. Se verificó decodificando un token real de las pruebas: <code>exp - iat = 28,800 s</code>
Variables de entorno	El secreto JWT_SECRET y la cadena de conexión a MongoDB se leen de <code>process.env</code> ; en el pipeline se usa un secreto de prueba y las URLs de despliegue se guardan como GitHub Secrets
Control de acceso	<code>verificarToken</code> responde 401 y <code>requireRol</code> responde 403 en cada ruta protegida
Transporte seguro	HTTPS en Vercel y Render; conexión cifrada con MongoDB Atlas

Evidencia: código del middleware

Backend/src/middleware/auth.js

```
const jwt = require('jsonwebtoken');

function verificarToken(req, res, next) {
  const token = req.headers['authorization']?.split(' ')[1];
  if (!token) return res.status(401).json({ mensaje: 'Token requerido' });

  try {
    req.usuario = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (error) {
    res.status(401).json({ mensaje: 'Token inválido o expirado' });
  }
}

module.exports = verificarToken;
```

Backend/src/middleware/requireRol.js

```
// Middleware de autorización por rol. Se usa DESPUÉS de verificarToken,
// porque depende de que req.usuario ya exista (con el rol incluido en el JWT).
function requireRol(...rolesPermitidos) {
  return (req, res, next) => {
    if (!req.usuario || !rolesPermitidos.includes(req.usuario.rol)) {
      return res.status(403).json({ mensaje: 'No tienes permiso para realizar esta acción' });
    }
    next();
  };
}
```

```
};
}

module.exports = requireRol;
```

En palabras sencillas: verificarToken funciona como un guardia en la puerta. Si la petición no trae token, o el token es falso o ya caducó, responde 401. Si es válido, guarda los datos del usuario y lo deja pasar. Después, requireRol revisa si el rol del usuario está en la lista de roles permitidos para esa ruta; si no lo está, responde 403.

Evidencia: pruebas en Postman

La colección *GestonHospitalaria API* se organiza en carpetas (1. Auth, 2. Usuarios, 3. Horarios, 4. Medicamentos, 5. Citas) e incluye pruebas positivas y negativas. Todas obtuvieron el resultado esperado:

Prueba	Esperado	Obtenido
Registro de paciente	201 + token	✓ 201 Created
Login admin, doctor y paciente	200 + token con el rol correcto	✓ 200 OK
Login con credenciales incorrectas	401	✓ 401 Unauthorized
Crear doctor sin token	401	✓ 401 "Token requerido"
Crear horario sin ser admin	403	✓ 403 Forbidden
Crear, listar y eliminar doctores (admin)	201 / 200	✓
Crear medicamento y actualizar stock (admin)	201 / 200	✓
Editar horario (admin) y consultar agenda (doctor)	200	✓

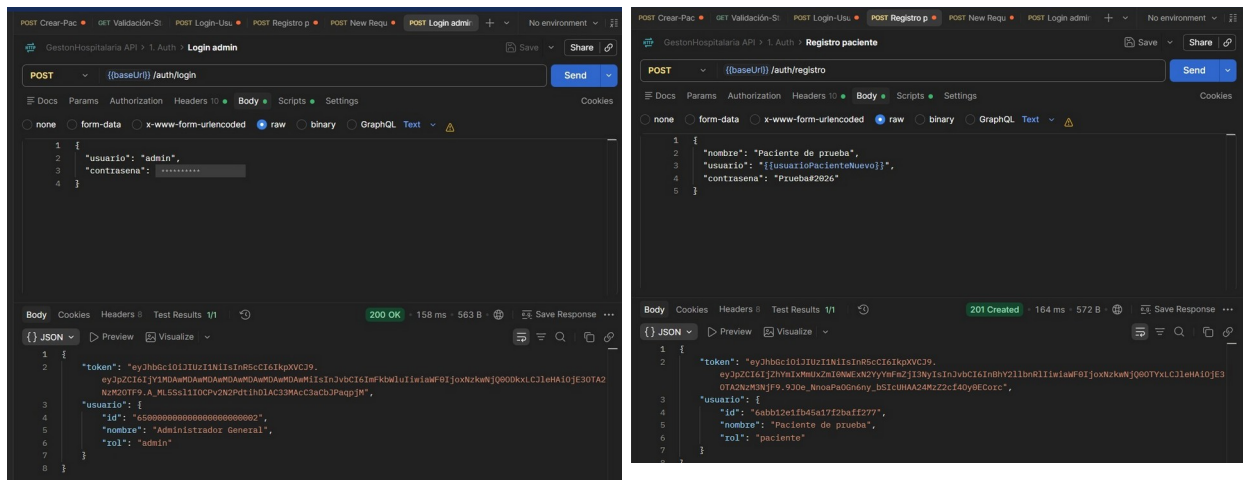


Figura 11. Postman: login del administrador (contraseña oculta) y registro de paciente con emisión de token.

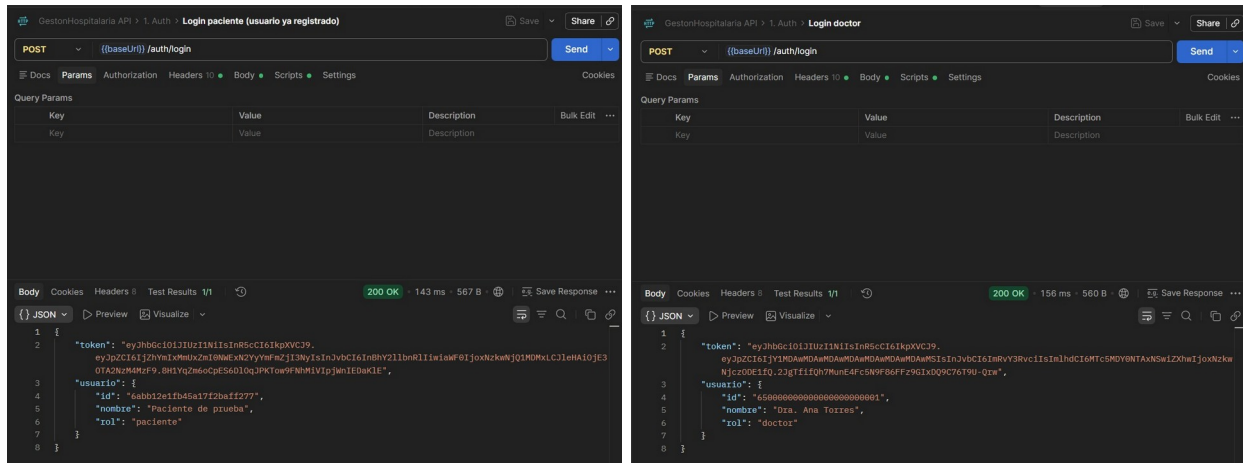


Figura 12. Postman: el token incluye el rol correcto para paciente y doctor.

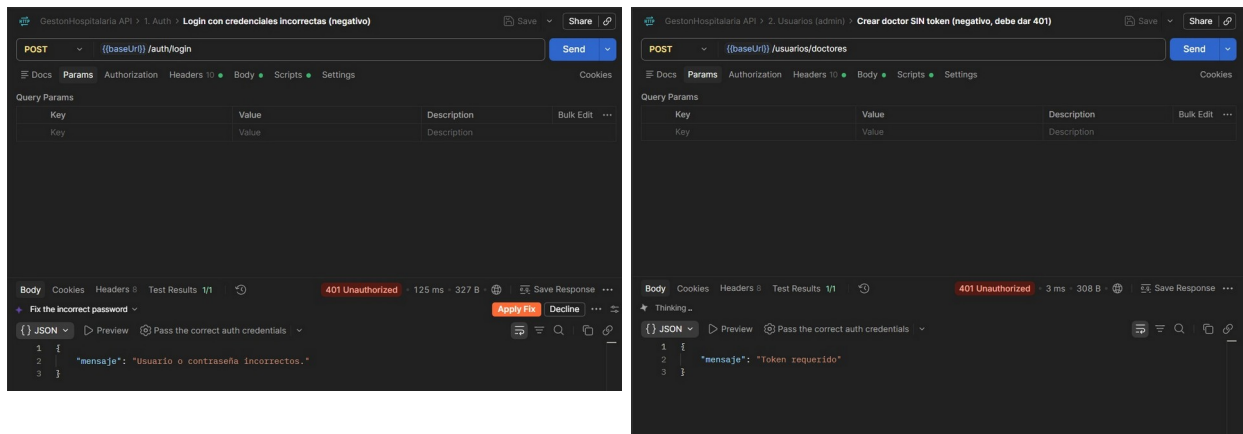


Figura 13. Postman: pruebas negativas con respuesta 401 (credenciales incorrectas y petición sin token).

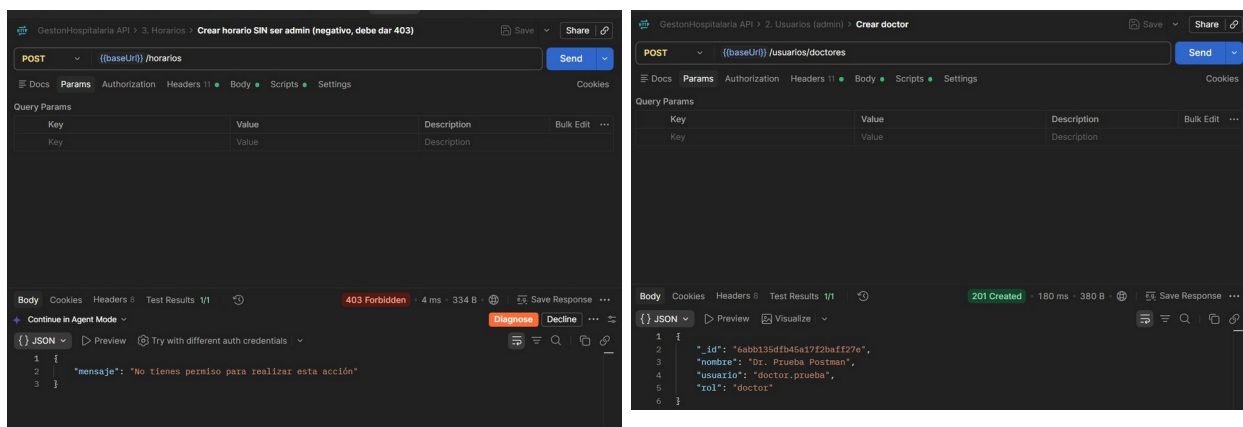


Figura 14. Postman: un paciente no puede crear horarios (403); el administrador sí puede crear doctores (201).

3.3 CI/CD con GitHub Actions

El pipeline CI/CD FarmaCitas está definido en `.github/workflows/ci.yml` y tiene cuatro trabajos (jobs).

Qué dispara el pipeline

- **Push a cualquier rama** (branches: ["*"]): cada cambio que sube un integrante se prueba automáticamente.
- **Pull request hacia `main`**: el código se valida antes de integrarse a la rama principal.
- **Despliegue y escaneo**: solo se ejecutan cuando el cambio llega a main (if: github.ref == 'refs/heads/main').

Etapas

Job	Cuándo corre	Qué hace
backend	Siempre	Descarga el código, instala Node.js 20, ejecuta <code>npm install</code> y corre las pruebas unitarias con Jest (<code>npm test</code>) usando un <code>JWT_SECRET</code> de prueba.
frontend	Siempre, en paralelo	Instala dependencias y compila la aplicación con <code>npm run build</code> (Vite). Si el código tiene errores, la compilación falla.
deploy	Solo en main, si backend y frontend pasaron	Activa el despliegue del backend en Render y del frontend en Vercel mediante deploy hooks, y espera 60 segundos a que ambos estén en línea.
owasp-zap	Solo en main, después del deploy	Ejecuta un escaneo baseline de OWASP ZAP en Docker sobre la URL publicada y guarda los reportes HTML y XML como artefacto <code>reporte-owasp-zap</code> .

Comparación con las etapas solicitadas: la instalación de dependencias, las pruebas unitarias y el despliegue a un ambiente en la nube se cumplen. El lint está configurado en el frontend (`eslint.config.js`), pero todavía no se ejecuta en el pipeline. La construcción de imágenes Docker se usa en el entorno local con `docker-compose`, mientras que en la nube Render y Vercel construyen la aplicación directamente desde el repositorio.

Flujo completo: push → pruebas del backend + build del frontend → (si es main y todo pasó) despliegue a Render y Vercel → escaneo OWASP ZAP → reporte guardado.

Evidencia

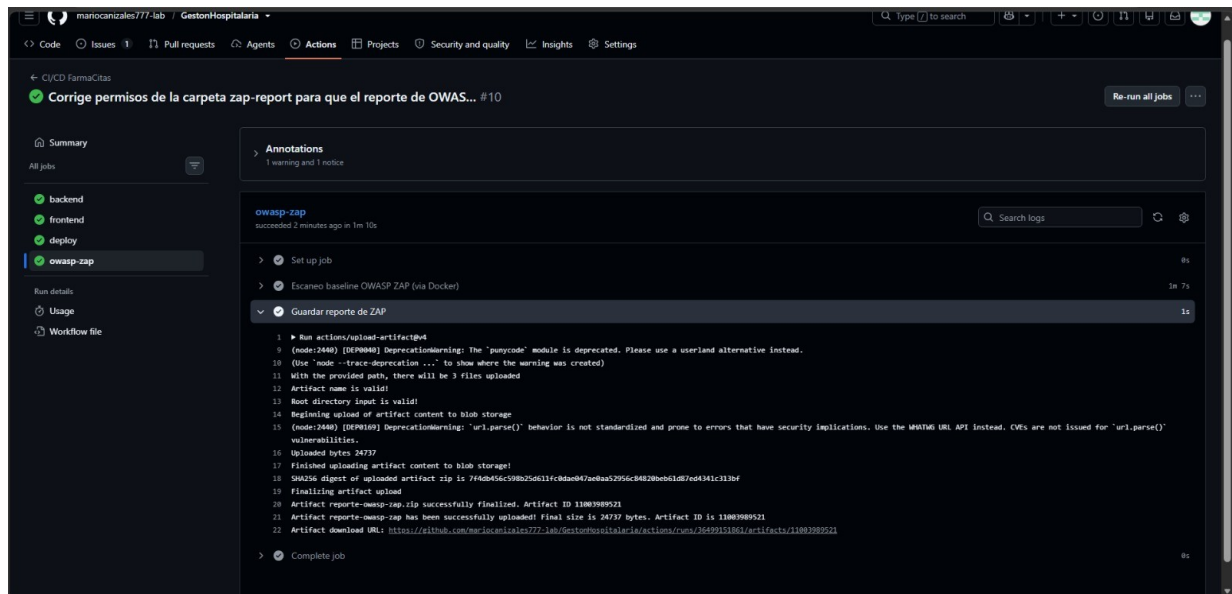


Figura 15. Ejecución #10 del pipeline en GitHub Actions: los cuatro jobs terminaron correctamente y se generó el artefacto del reporte ZAP.

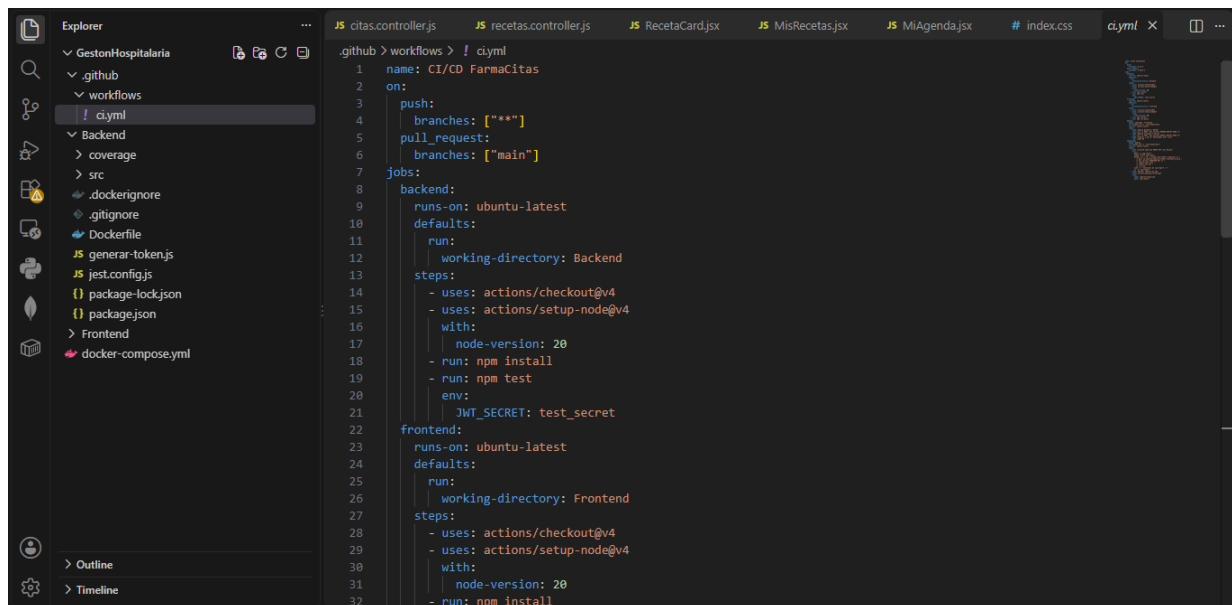


Figura 16. Archivo ci.yml en el repositorio: disparadores y jobs de backend y frontend.

```

33     - run: npm run build
34   deploy:
35     needs: [backend, frontend]
36     if: github.ref == 'refs/heads/main'
37     runs-on: ubuntu-latest
38     steps:
39       - name: Deploy Backend a Render
40         run: curl -X POST "${{ secrets.RENDER_DEPLOY_HOOK }}"
41       - name: Deploy Frontend a Vercel
42         run: curl -X POST "${{ secrets.VERCEL_DEPLOY_HOOK }}"
43       - name: Esperar a que el despliegue esté listo
44         run: sleep 60
45   owasp-zap:
46     needs: deploy
47     if: github.ref == 'refs/heads/main'
48     runs-on: ubuntu-latest
49     steps:
50       - name: Escaneo baseline OWASP ZAP (via Docker)
51         run: |
52           mkdir -p zap-report
53           chmod -R 777 zap-report
54           docker run --rm -v $(pwd)/zap-report:/zap/wrk:rw \
55             -t ghcr.io/zaproxy/zaproxy:stable zap-baseline.py \
56             -t "${{ secrets.FRONTEND_URL }}" \
57             -r report_html.html \
58             -x report_xml.xml \
59             -a || true
60         echo "--- Contenido de zap-report ---"
61         ls -la zap-report
62
63   - name: Guardar reporte de ZAP
64     uses: actions/upload-artifact@v4
65     with:
66       name: reporte-owasp-zap
67       path: zap-report/
    
```

Figura 17. Archivo ci.yml: jobs de despliegue (Render y Vercel) y escaneo OWASP ZAP.

En **Render**, cada despliegue queda registrado con su commit; el log confirma la conexión a MongoDB y el servicio en línea. En **Vercel**, además de los despliegues de producción desde main, se generan despliegues de vista previa (*Preview*) para cada rama de funcionalidad, como implementacionjest y pipeline-Frontend, antes de integrarlas mediante pull request.

DEPLOY	TRIGGER	DURATION
Mejoras en roles y seguridad 0248b47 - Deployed 18h ago	Deploy Hook	49.4s
Mejoras en roles y seguridad 0248b47 - Deployed 18h ago	Auto-Deploy	35.3s
Corrige job de OWASP ZAP corriendo el escaneo via Docker 19580c - Deployed 18h ago	Deploy Hook	35.7s
Corrige permisos de owasp-zap y agrega Dockerfile/docker-compose 8662226 - Deployed 18h ago	Deploy Hook	34.8s
Corrige permisos de owasp-zap y agrega Dockerfile/docker-compose 8662226 - Deployed 18h ago	Auto-Deploy	38.1s

Figura 18. Historial de despliegues del backend en Render, activados por el deploy hook del pipeline.

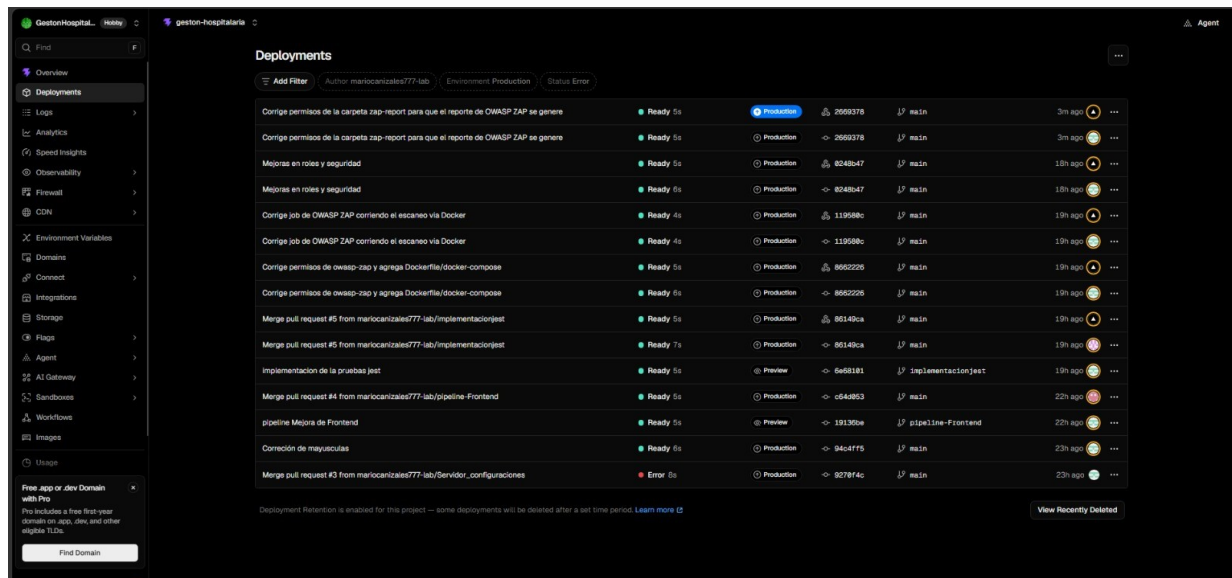


Figura 19. Historial de despliegues del frontend en Vercel (producción desde main y previews por rama).

URLs de los entornos

Frontend: <https://geston-hospitalaria.vercel.app>

Backend (API): <https://farmacitas-backend.onrender.com>

Nota: el backend usa el plan gratuito de Render, que suspende el servicio tras un periodo de inactividad; la primera petición puede tardar alrededor de 50 segundos. Por eso el pipeline espera 60 segundos antes del escaneo.

4. Pruebas y calidad

4.1 Pruebas unitarias y cobertura ($\geq 80\%$)

Framework	Jest, con reporte de cobertura de Istanbul (formatos HTML, lcov y clover); configuración en Backend/jest.config.js
Ejecución	Local con npm test y automática en el job backend del pipeline en cada push
Qué se probó	La lógica de negocio de los controladores de citas, pedidos y recetas (en esta arquitectura la lógica vive en los controladores), incluyendo flujos exitosos y caminos alternos o de error
Evidencia	Reporte HTML en Backend/coverage/lcov-report/index.html, generado el 28/09/2026

Resultados

Archivo	Sentencias	Ramas	Funciones	Líneas
citas.controller.js	89.36% (42/47)	81.81% (18/22)	100% (6/6)	93.02% (40/43)
pedidos.controller.js	96.36% (53/55)	93.54% (29/31)	100% (5/5)	98.00% (49/50)
recetas.controller.js	89.06% (57/64)	76.00% (19/25)	100% (7/7)	91.80% (56/61)
Total	91.56% (152/166)	84.61% (66/78)	100% (18/18)	94.15% (145/154)

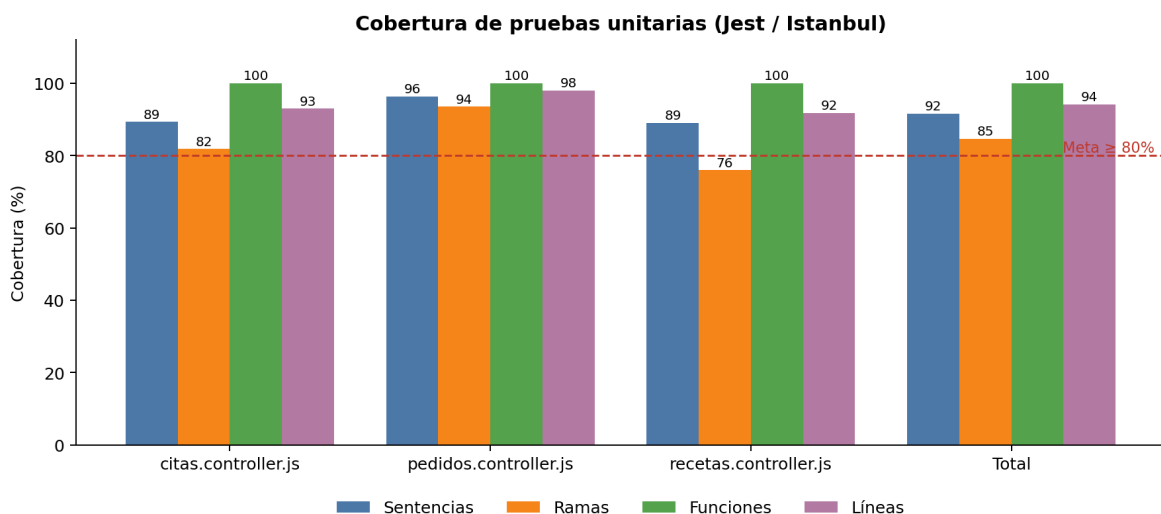


Figura 20. Cobertura de pruebas unitarias por archivo frente a la meta del 80%.

Interpretación: la cobertura supera la meta del 80% en todas las métricas. El 100% de funciones indica que cada función de los controladores se ejecuta al menos una vez en las pruebas. La cobertura de ramas (84.61%) es la más baja porque algunos caminos de error aún no se prueban, sobre todo en recetas.controller.js (76%).

Limitación: el alcance del reporte incluye tres controladores. Los módulos de autenticación, usuarios, horarios y medicamentos se validaron mediante las pruebas de API en Postman (sección 3.2), pero todavía no tienen pruebas unitarias automatizadas. Agregarlas es la primera acción del plan de mejora (sección 5.3).

4.2 Pruebas de seguridad: OWASP ZAP

Tipo de escaneo	ZAP Automated Baseline Scan (v2.17.0), ejecutado en Docker dentro del job owasp - zap del pipeline
URL objetivo	https://geston-hospitalaria.vercel.app
Evidencia	Artefacto reporte-owasp-zap (report_html.html y report_xml.xml) en GitHub Actions

Hallazgos principales

Hallazgo	Riesgo	Descripción
XSS (reflejado o almacenado)	—	No se detectaron alertas
Inyección SQL	—	No se detectaron alertas
Content Security Policy (CSP) Header Not Set	Medio	No se define la cabecera Content-Security-Policy, lo que deja al sitio sin una capa de protección contra inyección de código
Missing Anti-clickjacking Header	Medio	Falta X-Frame-Options o frame-ancestors; el sitio podría embeberse en un iframe malicioso
Cross-Domain Misconfiguration	Medio	Access-Control-Allow-Origin: * permitía peticiones desde cualquier dominio
X-Content-Type-Options Header Missing	Bajo	Falta la cabecera nosniff; expone a MIME-sniffing
Permissions Policy Header Not Set	Bajo	No se limita el uso de funciones del navegador (cámara, micrófono, geolocalización)
COEP y COOP Missing	Bajo	No hay aislamiento del contexto de navegación entre orígenes

Nota metodológica: el escaneo baseline es pasivo; revisa las respuestas del sitio pero no lanza ataques. Por eso, la ausencia de alertas de XSS o inyección no garantiza que no existan. Un escaneo activo (*full scan*) o un escaneo de API sobre el backend daría una evaluación más completa, en especial para inyección NoSQL en MongoDB.

Acciones tomadas

Acción	Detalle
Anti-clickjacking	Se configuró la cabecera X-Frame-Options: SAMEORIGIN
Protección anti-MIME sniffing	Se agregó X-Content-Type-Options: nosniff
Restricción de CORS	Se ajustó Access-Control-Allow-Origin para permitir solo dominios autorizados en lugar de *

Riesgos aceptados y pendientes

- **Aceptado:** alertas informativas de política de caché en los recursos estáticos de Vercel (CSS, JS, SVG), porque no contienen datos sensibles ni de sesión.
- **Aceptado temporalmente:** ausencia de COOP y COEP (riesgo bajo).
- **Pendiente:** definir Content-Security-Policy y Permissions-Policy (incluidas en el plan de mejora de corto plazo).

4.3 Calidad de código: SonarQube

El proyecto se analizó en **SonarQube Cloud** sobre la rama `main` (5.9k líneas de código) con el Quality Gate estándar `*Sonar way*`.

Métrica	Código global	Código nuevo
Quality Gate	—	No aprobado (3 condiciones fallidas)
Seguridad (vulnerabilidades)	18 issues abiertos – calificación E	Calificación E (requerida: A)
Confiabilidad (bugs)	387 issues abiertos – calificación C	Calificación C (requerida: A)
Mantenibilidad (code smells)	764 issues abiertos – calificación A	763 issues nuevos
Duplicación	12.1% (sobre 6.7k líneas)	16.16% sobre 3.9k líneas nuevas (límite ≤ 3.0%)
Cobertura	No configurada	No configurada
Deuda técnica	Representada por los 764 code smells	—

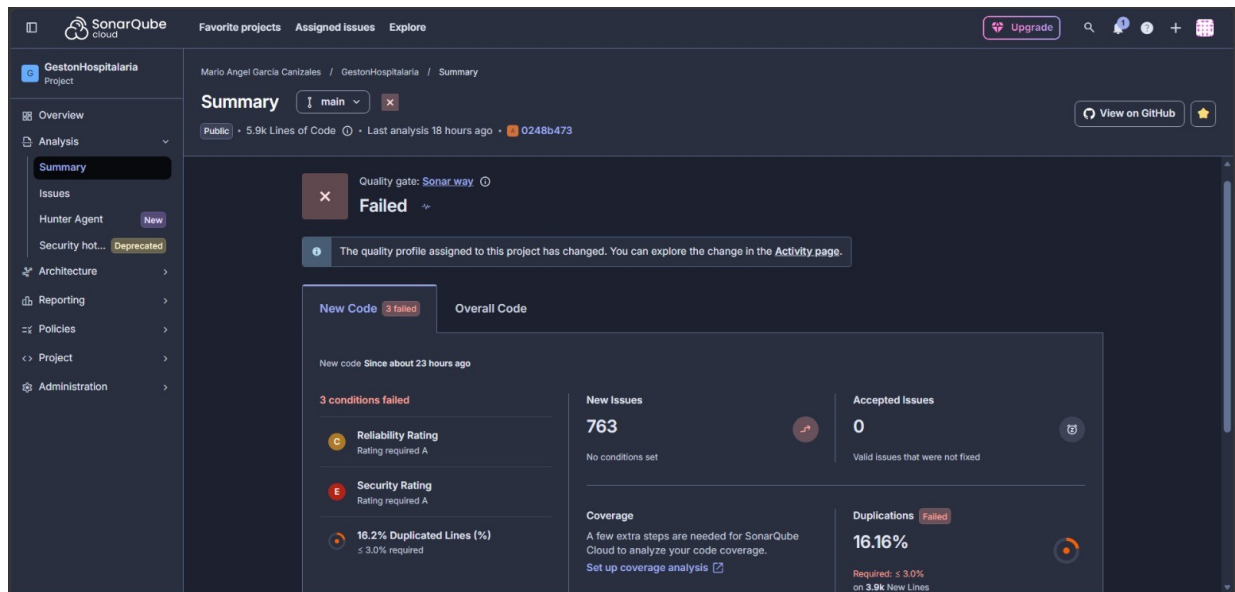


Figura 21. SonarQube Cloud: Quality Gate en código nuevo.

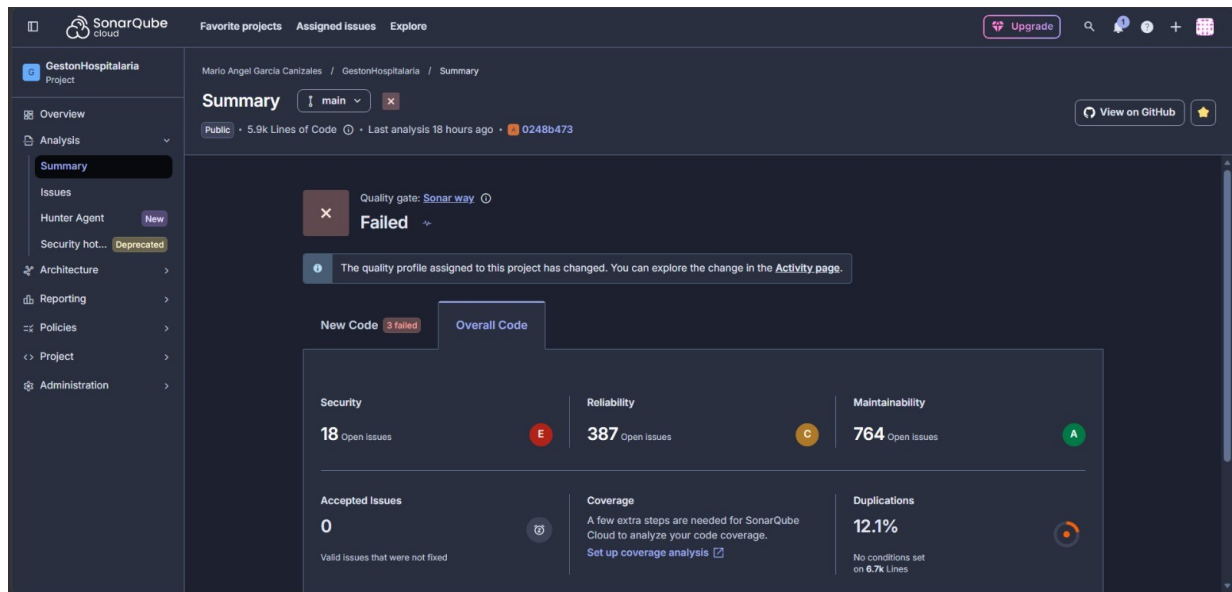


Figura 22. SonarQube Cloud: métricas del código global.

Interpretación y acciones

- **Mantenibilidad A:** a pesar del número de code smells, la deuda técnica es baja en proporción al tamaño del código.
- **Seguridad E y confiabilidad C:** son las métricas que hacen fallar el Quality Gate. Cada uno de los 18 issues de seguridad debe revisarse en la pestaña *Issues* y corregirse o justificarse.
- **Duplicación y volumen de issues:** el repositorio incluye archivos generados, como la carpeta Backend/coverage/lcov-report (HTML y JavaScript del reporte). Si SonarQube los analiza, inflan la duplicación y el número de issues. El primer paso es excluirlos con `sonar.exclusions` y volver a medir.
- **Cobertura:** el reporte `lcov.info` de Jest ya existe; basta con importarlo mediante `sonar.javascript.lcov.reportPaths` para que SonarQube muestre la cobertura real.

5. Cierre y evaluación

5.1 Comparación: planificado vs. ejecutado

Alcance y cronograma

HU	Planificado	Ejecutado	Estado	Desviación
HU3 – Gestión de horarios	24 – 29 ago	24 ago – 9 sept	✓ Done	+11 días
HU1 – Agendamiento de citas	7 – 18 sept	9 – 18 sept	✓ Done	Inicio +2 días; cierre en fecha
HU4 / HU6 – Autenticación	No planeada	10 – 16 sept	✓ Done	Alcance nuevo
HU5 – Medicamentos y recetas	No planeada	17 – 21 sept	✓ Done	Alcance nuevo
HU7 – Interfaz, pedidos y reportes	No planeada	22 – 25 sept	✓ Done	Alcance nuevo
HU2 – Consulta y administración	21 sept – 2 oct	21 – 27 sept	✓ Done	5 días antes

Decisiones técnicas: diseño vs. implementación

Aspecto	Planificado (Act. 4)	Ejecutado	Motivo
Arquitectura	Microservicios	Monolito modular (módulos 1:1 con los microservicios)	Tamaño del equipo y 6 semanas de duración
Base de datos	PostgreSQL	MongoDB Atlas	Decisión del equipo; esquemas flexibles
Infraestructura	Kubernetes + HPA	Vercel + Render	Servicios gestionados, sin administrar servidores
Orquestación local	Docker Compose	Docker Compose	Se mantuvo para desarrollo
Mensajería	RabbitMQ	No implementada	Sin necesidad de comunicación asíncrona en el alcance actual
Reserva	Bloqueo temporal de 10 min	Cambio directo disponible → ocupado	Simplificación del flujo
Recordatorios	SMS / correo 1 h antes	Recordatorios en la interfaz	Sin proveedor externo contratado
Medicamentos	Sugerencia de genéricos	Recetas + control de medicamentos controlados	Se priorizó la trazabilidad de controlados
Seguridad	JWT + bcrypt + HTTPS	JWT (8 h) + bcrypt + requireRol + HTTPS + ZAP + SonarQube	Se cumplió y se amplió
CI/CD	Build → test → Docker → registry	Test → build → deploy → escaneo ZAP	Despliegue directo a la nube
Monitoreo	Prometheus + Grafana	Logs y métricas de Render y Vercel	Herramientas incluidas en las plataformas

Indicadores medibles

Indicador	Meta	Resultado	Cumple
Historias de usuario completadas	3 planeadas	7 (3 planeadas + 4 surgidas)	✓
Criterios de aceptación (checklists de Trello)	100%	100% en las 7 HU	✓
Cobertura de líneas	≥ 80%	94.15%	✓
Cobertura de funciones	≥ 80%	100% (18/18)	✓
Pruebas de API en Postman	Casos positivos y negativos	15 casos, todos con el resultado esperado	✓
Pipeline CI/CD con despliegue automático	Funcionando	4 jobs en verde (ejecución #10)	✓
Alertas de riesgo alto en OWASP ZAP	0	0 (3 de riesgo medio atendidas)	✓
Quality Gate de SonarQube	Aprobado	No aprobado (seguridad E, confiabilidad C, duplicación 16.16%)	✗
Desviación máxima del cronograma	±10%	HU3 +11 días; el proyecto cerró dentro de las 6 semanas	Parcial

5.2 Lecciones aprendidas

Planeación

- **La estimación de la historia base fue muy optimista.** Gestión de horarios se planeó en 6 días y tomó 17. Como todo el sistema depende de ella, su retraso movió el inicio de las demás; debió tener más margen.
- **El plan original solo contempló funciones visibles para el paciente.** La autenticación, las recetas y las interfaces por rol (HU4 a HU7) aparecieron durante la ejecución. Planear desde el inicio el trabajo transversal (seguridad, interfaz, pipeline) habría dado un cronograma más realista.
- **Los criterios de aceptación deben actualizarse cuando cambia el diseño.** El checklist de HU4 menciona validar la unicidad del correo, pero el sistema usa nombre de usuario; el de HU5 permite varios medicamentos por receta, pero el modelo guarda uno.

Arquitectura

- **Microservicios era demasiado para 5 personas en 6 semanas.** El monolito modular permitió entregar a tiempo y, como cada módulo corresponde a un microservicio planeado, la migración futura sigue siendo posible.
- **Cambiar de base de datos a mitad del proyecto dejó documentación inconsistente.** El paso de PostgreSQL a MongoDB obligó a corregir diagramas y textos al final; las decisiones técnicas grandes deben cerrarse y registrarse en un solo lugar.
- **Los servicios gestionados simplificaron el despliegue.** Vercel, Render y MongoDB Atlas evitaron administrar servidores o un clúster de Kubernetes.

Calidad

- **Las pruebas negativas hicieron visible la seguridad.** Probar a propósito los casos 401 y 403 en Postman demostró que los permisos por rol funcionan, algo que las pruebas del "camino feliz" no muestran.
- **Una cobertura alta no garantiza que se cubra lo más crítico.** El 94% de líneas se concentra en citas, pedidos y recetas; la autenticación, que protege todo el sistema, no tiene pruebas unitarias.
- **Configurar una herramienta no es lo mismo que usarla.** ESLint está configurado pero no se ejecuta en el pipeline, y SonarQube se integró al final, cuando ya había cientos de issues acumulados.

Seguridad

- **Separar autenticación y autorización mantuvo el control de acceso consistente.** Tener `verificarToken` y `requireRol` como piezas independientes permite proteger cada ruta con una sola línea.
- **Configurar las cabeceras de seguridad desde el principio evita hallazgos en ZAP.** Las alertas de CSP, clickjacking y CORS abierto se corrigieron después del escaneo; con una configuración base desde el inicio no habrían aparecido.
- **Un escaneo que nunca falla solo informa.** El job de ZAP no detiene el pipeline (`|| true`); para que proteja realmente debe bloquear el despliegue ante riesgos altos.
- **La seguridad incluye cómo se comparte la evidencia.** La contraseña del administrador apareció en una captura de pruebas y tuvo que ocultarse antes de incluirla en este informe.

Gestión

- **Los checklists de Trello funcionaron como criterios de aceptación verificables.** Permitieron saber con claridad cuándo una historia estaba terminada.
- **El tablero debe reflejar el trabajo en tiempo real.** Las historias HU4 a HU7 se incorporaron al tablero después de iniciarse su desarrollo y algunas fechas se ajustaron al final, por lo que Trello marca "plazo vencido" en HU1 y HU3.
- **Las ramas por funcionalidad y los pull requests ordenaron la integración.** Cada funcionalidad (por ejemplo, las pruebas con Jest o el pipeline del frontend) se probó en su rama con despliegues de vista previa antes de llegar a `main`.
- **Los roles definidos desde la Carta del Proyecto evitaron duplicar trabajo.** Cada integrante sabía qué le correspondía.

5.3 Plan de mejora continua

Corto plazo (1 - 2 semanas)

Acción	Meta medible
Pruebas unitarias y de integración para autenticación y usuarios (Jest + Supertest)	Cobertura $\geq$ 80% en <code>auth.controller</code> y <code>usuarios.controller</code>
Limitar intentos de inicio de sesión (<code>express-rate-limit</code>) con bloqueo temporal	Máximo 5 intentos fallidos cada 15 minutos por usuario
Validar y limpiar datos de entrada (<code>express-validator</code>), incluida protección contra inyección NoSQL	100% de los endpoints POST y PUT con validación

Acción	Meta medible
Completar cabeceras de seguridad: Content-Security-Policy y Permissions-Policy (helmet)	0 alertas de riesgo medio en el siguiente escaneo ZAP
Agregar <code>npm run lint</code> al job de frontend del pipeline	El pipeline falla si hay errores de lint
Excluir archivos generados de SonarQube e importar <code>lcov.info</code>	Duplicación $\leq 3\%$ y cobertura visible en SonarQube
Alinear criterios de aceptación y modelos (recetas con varios medicamentos, usuario vs. correo)	0 diferencias entre los checklists de Trello y el código
Limpieza: eliminar <code>reportes.contrpller.js</code> duplicado, cambiar la contraseña del administrador del seed y desactivar el doble despliegue (auto-deploy + deploy hook)	Un despliegue por commit y ninguna credencial expuesta

Mediano plazo (1 – 2 meses)

Acción	Meta medible
Corregir los issues de seguridad y confiabilidad de SonarQube	Quality Gate aprobado (seguridad A, confiabilidad A)
Implementar el bloqueo temporal de 10 minutos (estado <code>bloqueado</code> con expiración automática en MongoDB)	0 reservas dobles en pruebas de concurrencia
Enviar recordatorios reales por correo 1 hora antes de la cita	Medir y reducir el porcentaje de citas no atendidas
Registro de auditoría: quién creó, modificó o canceló citas, horarios y recetas	100% de las acciones administrativas registradas
Análisis SAST (CodeQL, <code>npm audit</code>) y DAST (ZAP full scan y API scan) que bloqueen el despliegue ante riesgos altos	0 vulnerabilidades altas en producción
Ampliar el módulo de reportes con un dashboard (ocupación por sucursal, cancelaciones, stock bajo)	Dashboard disponible para el administrador

Largo plazo (3 – 6 meses)

Acción	Meta medible
IA para predecir la demanda de citas por sucursal, día y horario (series de tiempo)	Abrir horarios según la demanda esperada
IA para predecir inasistencias y reforzar recordatorios a pacientes con mayor riesgo	Reducir el porcentaje de citas no atendidas
Predicción de stock de medicamentos por temporada (por ejemplo, antigripales en invierno)	Menos medicamentos agotados
Observabilidad: métricas, trazas y alertas (Prometheus/Grafana o Sentry)	Detectar errores antes de que los reporten los usuarios
Plan de pago en Render o infraestructura propia para eliminar la suspensión por inactividad	Tiempo de respuesta < 2 s en la primera petición
Migración gradual a microservicios si la carga lo justifica, empezando por citas y horarios	Escalar solo los módulos con mayor demanda

6. Anexos (evidencias)

A1. Carta del Proyecto (Actividad 1)

Síntesis en la sección 2.1. Directorio del equipo:

Nombre	Matrícula	Rol
Mario Ángel García Canizales	3065072	Coordinador del Proyecto
Pablo Ángel Garza Ramírez	7094461	Administrador de Base de Datos y QA
Rubén Miguel Manzanares Tovar	2886925	Front-end
Christopher Abraham Chairez Cruz	3038149	Back-end
Luis Carlos Soto Carrillo	7039730	Analista y Documentador

A2. Planeación: historias de usuario y Trello (Actividad 3)

Síntesis en la sección 2.2. Tarjetas con sus checklists completados:

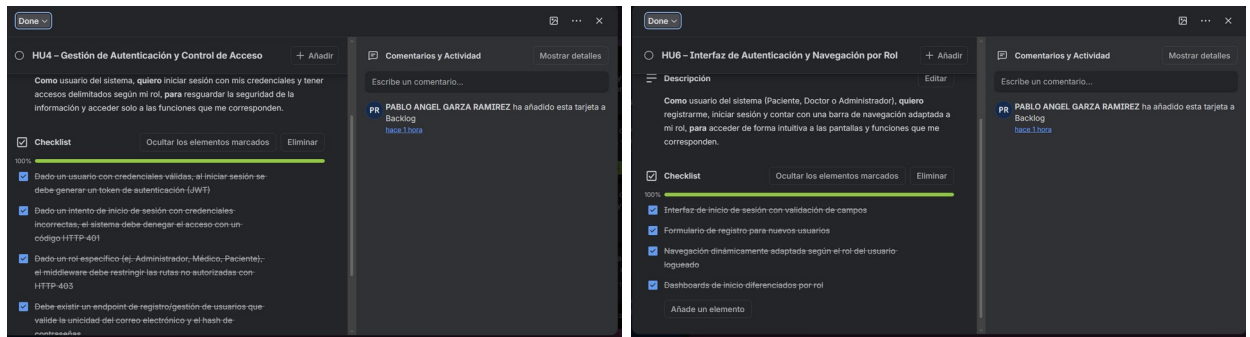


Figura 23. Tarjetas HU4 (autenticación y control de acceso) y HU6 (interfaz de autenticación por rol).

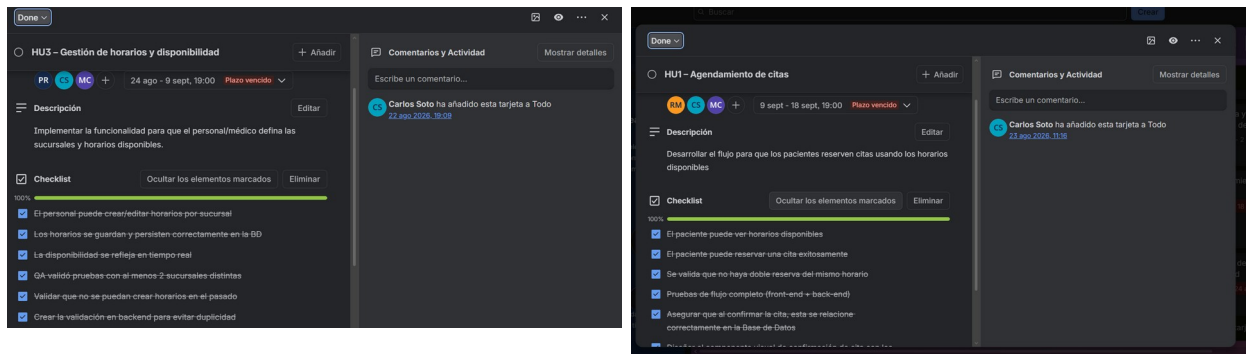


Figura 24. Tarjetas HU3 (gestión de horarios) y HU1 (agendamiento de citas).

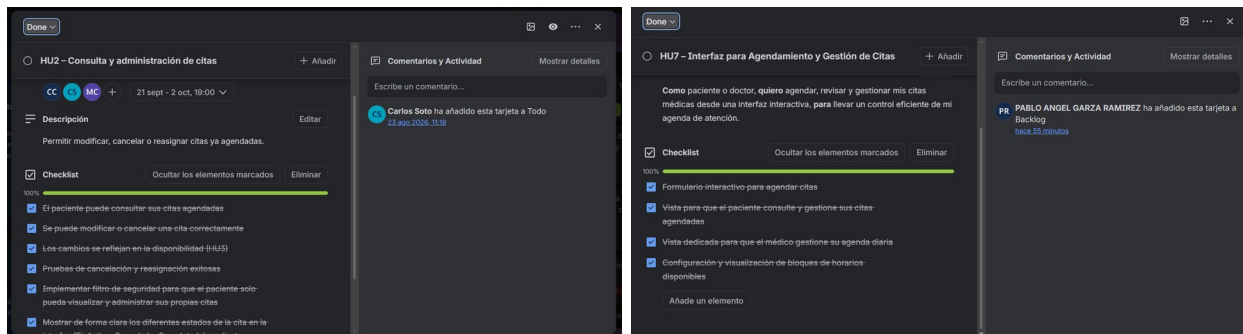


Figura 25. Tarjetas HU2 (consulta y administración de citas) y HU7 (interfaz de agendamiento).

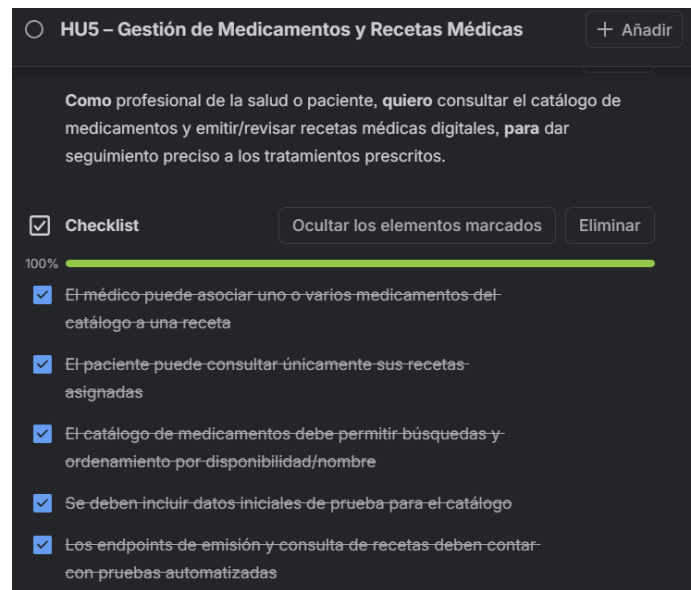


Figura 26. Tarjeta HU5 (medicamentos y recetas médicas).

A3. Arquitectura UML y 4+1 (Actividad 4)

Diagramas en la sección 2.3 (figuras 4 a 10). Estructura real del repositorio y modelos de datos:

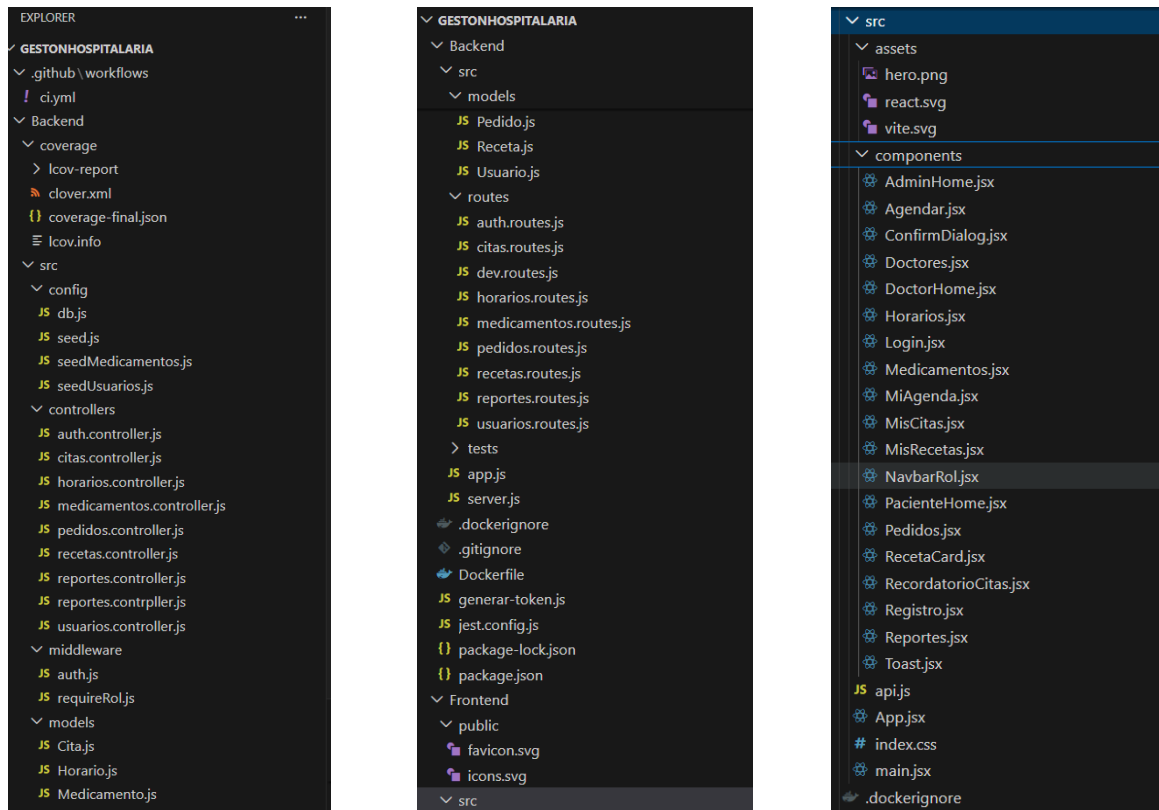


Figura 27. Estructura del repositorio GestonHospitalaria: backend (config, controllers, middleware, models, routes, tests) y frontend (components).

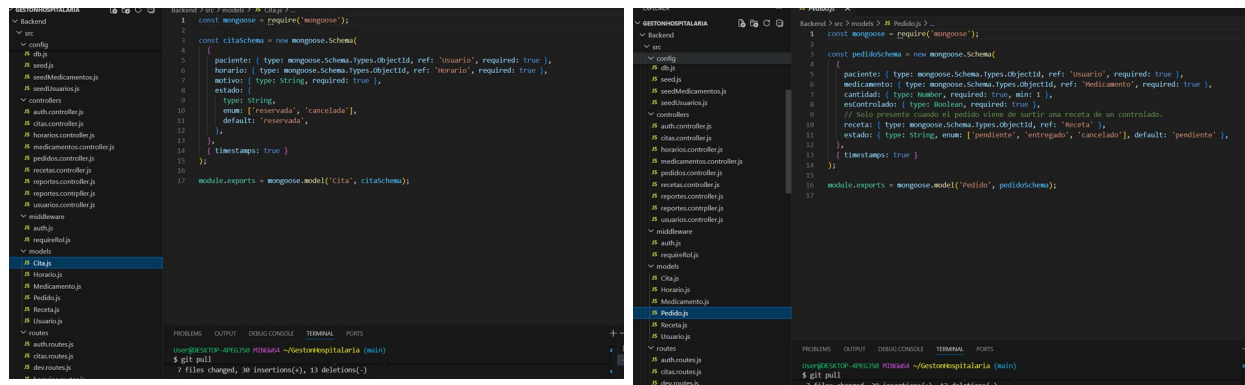


Figura 28. Modelos Mongoose de Cita y Pedido.

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'GESTONHOSPITALARIA' with folders like 'Backend', 'src', 'config', 'controllers', 'middleware', 'models', 'routes', and 'utils'. The 'models' folder is expanded, showing 'Receta.js' selected. The code editor shows the following code:

```

1  const mongoose = require('mongoose');
2
3  const recetaSchema = new mongoose.Schema(
4
5    {
6      cita: { type: mongoose.Schema.Types.ObjectId, ref: 'cita', required: true },
7      paciente: { type: mongoose.Schema.Types.ObjectId, ref: 'Usuario', required: true },
8      emitidoPor: { type: mongoose.Schema.Types.ObjectId, ref: 'Usuario', required: true },
9      medicamento: { type: mongoose.Schema.Types.ObjectId, ref: 'Medicamento', required: true },
10     cantidad: { type: Number, required: true, min: 1 },
11     esControlado: { type: Boolean, required: true },
12   },
13   { timestamps: true }
14 );
15
16 module.exports = mongoose.model('Receta', recetaSchema);
    
```

The terminal at the bottom shows the command prompt for 'MINGW64 ~\GestonHospitalaria (main)' and the output of 'git pull', which indicates that 7 files changed, 30 insertions(+), and 13 deletions(-).

Figura 29. Modelo Mongoose de Receta.

A4. Reporte de cobertura (Jest)

Resultados en la sección 4.1. Reporte completo en Backend/coverage/lcov-report/index.html (también disponible en coverage/lcov.info, clover.xml y coverage-final.json).

A5. Reporte OWASP ZAP

Hallazgos y acciones en la sección 4.2. El reporte completo se descarga desde el artefacto reporte-owasp-zap de la ejecución del pipeline en GitHub Actions.

A6. Reporte SonarQube

Métricas en la sección 4.3 (figuras del análisis de código nuevo y global en SonarQube Cloud).

A7. Evidencias CI/CD

Capturas del pipeline en la sección 3.3. Log del despliegue del backend y pruebas adicionales de API:

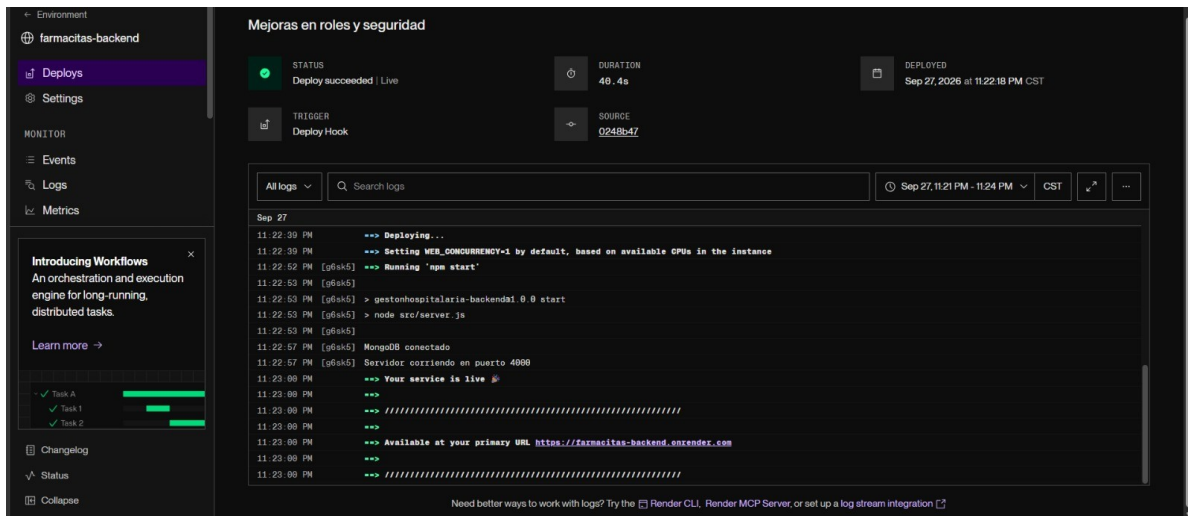


Figura 30. Log de despliegue en Render: servidor iniciado, MongoDB conectado y servicio en línea.

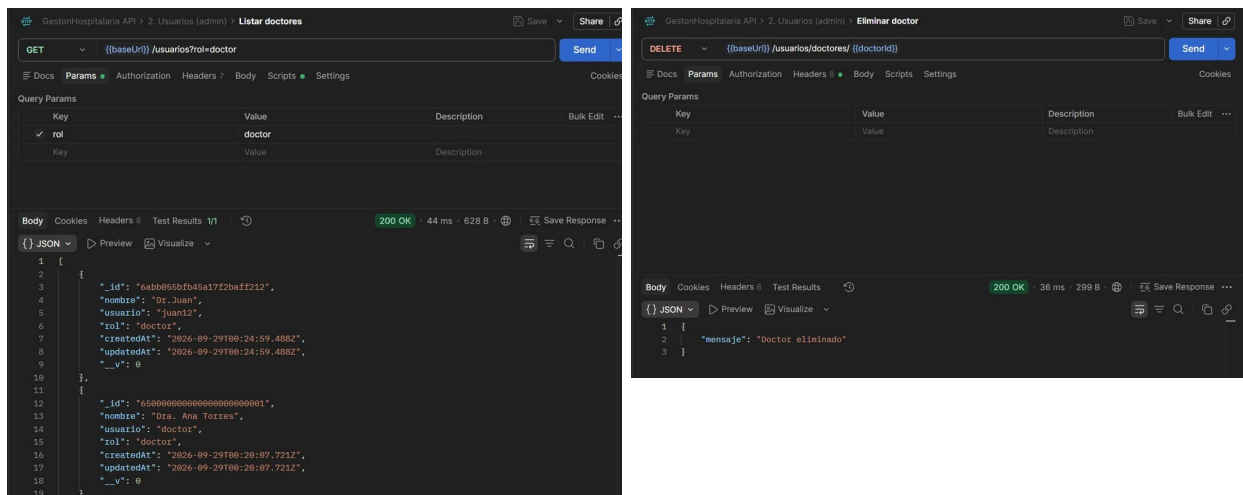


Figura 31. Postman: listar doctores y eliminar doctor (administrador).

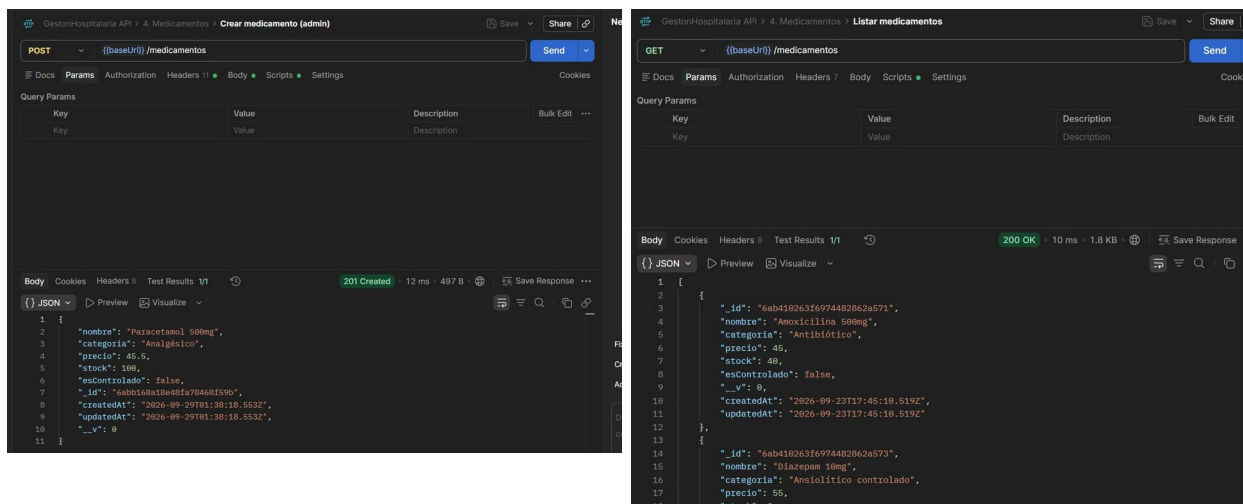


Figura 32. Postman: crear medicamento (administrador) y listar catálogo.

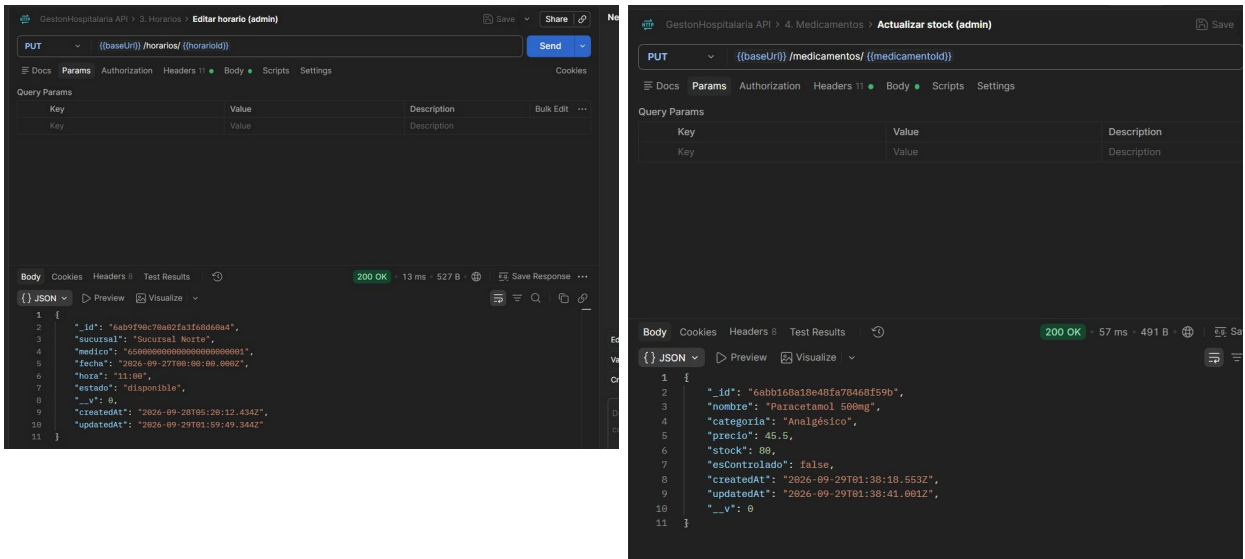


Figura 33. Postman: editar horario y actualizar stock (administrador).

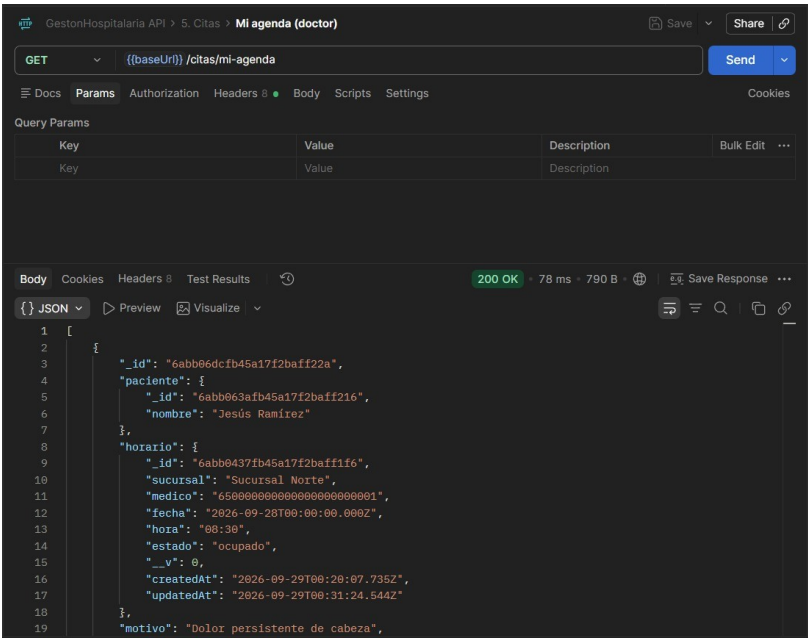


Figura 34. Postman: agenda del doctor con los datos de la cita y del paciente.

Archivo completo `.github/workflows/ci.yml`

```
name: CI/CD FarmaCitas
on:
  push:
    branches: ["**"]
  pull_request:
    branches: ["main"]
jobs:
  backend:
    runs-on: ubuntu-latest
    defaults:
      run:
```

```

    working-directory: Backend
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
    - run: npm install
    - run: npm test
    env:
      JWT_SECRET: test_secret
frontend:
  runs-on: ubuntu-latest
  defaults:
    run:
      working-directory: Frontend
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
    - run: npm install
    - run: npm run build
deploy:
  needs: [backend, frontend]
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - name: Deploy Backend a Render
      run: curl -X POST "${{ secrets.RENDER_DEPLOY_HOOK }}"
    - name: Deploy Frontend a Vercel
      run: curl -X POST "${{ secrets.VERCEL_DEPLOY_HOOK }}"
    - name: Esperar a que el despliegue esté listo
      run: sleep 60
owasp-zap:
  needs: deploy
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - name: Escaneo baseline OWASP ZAP (via Docker)
      run: |
        mkdir -p zap-report
        chmod -R 777 zap-report
        docker run --rm -v $(pwd)/zap-report:/zap/wrk/:rw \
          -t ghcr.io/zaproxy/zaproxy:stable zap-baseline.py \
          -t "${{ secrets.FRONTEND_URL }}" \
          -r report_html.html \
          -x report_xml.xml \
          -a || true
        echo "--- Contenido de zap-report ---"
        ls -la zap-report
    - name: Guardar reporte de ZAP
      uses: actions/upload-artifact@v4
      with:

```

```
name: reporte-owasp-zap
path: zap-report/
```

Repositorio y entornos

Repositorio: <https://github.com/mariocanizales777-lab/GestonHospitalaria>

Frontend (Vercel): <https://geston-hospitalaria.vercel.app>

Backend (Render): <https://farmacitas-backend.onrender.com>

Checklist de cumplimiento

Requisito	Estado	Evidencia
JWT implementado + expiración + secreto en variables de entorno	✓	Sección 3.2 (token de 8 h, <code>process.env.JWT_SECRET</code>)
Roles admin/usuario funcionando (rutas protegidas)	✓	Sección 3.2 (<code>requireRole</code> , pruebas 401/403 en Postman)
Pruebas unitarias con $\geq 80\%$ (reporte incluido)	✓	Sección 4.1 (94.15% de líneas) y anexo A4
Pipeline CI/CD funcionando (YAML + evidencias)	✓	Sección 3.3 y anexo A7
Despliegue automático a entorno de prueba	✓	Vercel y Render (sección 3.3)
OWASP ZAP ejecutado (reporte incluido)	✓	Sección 4.2 y anexo A5
SonarQube ejecutado (métricas documentadas)	✓	Sección 4.3 y anexo A6
Informe final con plan vs. ejecución, lecciones y mejora continua	✓	Secciones 5.1, 5.2 y 5.3
Actividades 1, 3 y 4 integradas explícitamente	✓	Secciones 2.1, 2.2 y 2.3; anexos A1 a A3

Bibliografía

- García Canizales, M. A., Garza Ramírez, P. A., Manzanares Tovar, R. M., Chairez Cruz, C. A., & Soto Carrillo, L. C. (2026). GestonHospitalaria [Repositorio de código]. GitHub.
<https://github.com/mariocanizales777-lab/GestonHospitalaria>
- GitHub. (s. f.). GitHub Actions documentation. <https://docs.github.com/actions>
- Jest. (s. f.). Getting started. <https://jestjs.io/docs/getting-started>
- Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>
- Kruchten, P. B. (1995). The 4+1 view model of architecture. IEEE Software, 12(6), 42–50.
<https://doi.org/10.1109/52.469759>
- Mongoose. (s. f.). Mongoose documentation. <https://mongoosejs.com/docs/>
- OWASP Foundation. (s. f.). ZAP – Baseline scan. <https://www.zaproxy.org/docs/docker/baseline-scan/>
- Schwaber, K., & Sutherland, J. (2020). La Guía de Scrum. <https://scrumguides.org/>
- SonarSource. (s. f.). SonarQube Cloud documentation. <https://docs.sonarsource.com/sonarqube-cloud/>